Crazy Stocks

Source code excerpt (first 30 + last 30 pages)

Scope: Assets_Game own source. Files: 44 / Lines: 14175 / 50 lines per page / Total pages: 284

Generated: 2026-10-14 11:59:26 (local build chain, headless Edge print)

```
// ===== FILE: Assets\_Game\Scripts\Editor\ArtCheck.cs =====
// ArtCheck.cs - the T019 art-intake gate (batchmode entry):
// Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyStocks.ArtCheck.Run -logFile <abs>
// Section1: catalog_foreign_key_law - when the ArtCatalog asset exists,
//            every key sits inside the frozen §7 closed set (15 stems,
//            G10-A01..A15); missing stems are LEGAL pre-arrival (honest
//            present/missing counts), foreign stems are RED.
// Section2: closed_set_parity_law - the TradeArt registry holds exactly
//            the fifteen frozen §7 stems, unique, with the anchor
//            faces (hall day, suoha portrait, ui kit) in place.
// Section3: trader_map_law - the four known comedy traders map to the
//            frozen §7 portraits and unmapped ids return null (the
//            procedural placeholder owns their face).
// Section4: null_safe_law - empty/null/missing lookups return null and
//            never throw on the headless host (X027 headless law).
// Section5: fallback_face_law - with the catalog absent every wired
//            face still resolves a procedural sprite (art-first,
//            fallback-always - views render on any project state).
// Exit codes: 0=all assertions pass  1=fail/exception
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using Sy.Core;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyStocks
{
    public static class ArtCheck
    {
        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;

        const string CatalogPath = "Assets/_Game/Resources/ArtCatalog.asset";

        public static void Run()
        {
            int code;
            try
            {
                Section1CatalogForeignKeyLaw();
                Section2ClosedSetParityLaw();
                Section3TraderMapLaw();
                Section4NullSafeLaw();
                Section5FallbackFaceLaw();
                code = _failed == 0 ? 0 : 1;
            }
            catch (Exception e)
```

```
        {
            Lines.Add("CRASH - " + e.Message);
            code = 1;
        }
        WriteReport();
        EditorApplication.Exit(code);
    }

    static void PassFail(string name, bool ok, string detail)
    {
        if (ok) _passed++; else _failed++;
        Lines.Add("[" + (ok ? "PASS" : "FAIL") + "] " + name + " - " + detail);
    }

    static void Section1CatalogForeignKeyLaw()
    {
        var catalog = AssetDatabase.LoadAssetAtPath<ArtCatalog>(CatalogPath);
        if (catalog == null)
        {
            PassFail("S1.catalog_absent_legal", true,
                "no ArtCatalog yet (pre-arrival state is legal)");
            return;
        }
        var closed = new HashSet<string>(TradeArt.AllStems);
        int foreign = 0, present = 0;
        foreach (var e in catalog.entries)
        {
            if (e == null || string.IsNullOrEmpty(e.key)) continue;
            if (!closed.Contains(e.key)) { foreign++; Lines.Add("FAIL detail - foreign key: " + e.key); }
            else if (e.sprite != null) present++;
        }
        PassFail("S1.zero_foreign_keys", foreign == 0,
            "entries=" + catalog.entries.Count + " present=" + present
            + "/" + TradeArt.AllStems.Length + " foreign=" + foreign
            + " (missing stems are legal until the S2.5 delivery lands)");
    }

    static void Section2ClosedSetParityLaw()
    {
        var stems = TradeArt.AllStems;
        bool count = stems.Length == 15;
        PassFail("S2.closed_set_size", count, "stems=" + stems.Length + " (expected 15, §7 G10-A01..A15)");

        var seen = new HashSet<string>();
        bool unique = true;
        foreach (var s in stems) if (!seen.Add(s)) unique = false;
        PassFail("S2.closed_set_unique", unique, "zero duplicate stems");

        bool anchors = Array.IndexOf(stems, TradeArt.HallDay) >= 0
            && Array.IndexOf(stems, TradeArt.TraderSuoha) >= 0
```

```
        && Array.IndexOf(stems, TradeArt.UiKit) >= 0
        && Array.IndexOf(stems, TradeArt.EventCardFrame) >= 0;
    PassFail("S2.anchor_stems_present", anchors,
        "hall_day + trd_suoha + ui_kit + eventcard_frame all registered");
}

static void Section3TraderMapLaw()
{
    bool known = TradeArt.TraderPortraitStem("trd_all_in_bro") == TradeArt.TraderSuoha
        && TradeArt.TraderPortraitStem("trd_top_escape_sis") == TradeArt.TraderTaoding
        && TradeArt.TraderPortraitStem("trd_counter_li") == TradeArt.TraderLaoli
        && TradeArt.TraderPortraitStem("trd_granny_info") == TradeArt.TraderDama;
    PassFail("S3.known_traders_map", known, "the four comedy faces hit their §7 portraits");

    bool unmapped = TradeArt.TraderPortraitStem("trd_steady_wang") == null
        && TradeArt.TraderPortraitStem("trd_sweeper_monk") == null
        && TradeArt.TraderPortraitStem("bogus_id") == null
        && TradeArt.TraderPortraitStem(null) == null;
    PassFail("S3.unmapped_null", unmapped,
        "no frozen portrait -> null (procedural owns the face)");
}

static void Section4NullSafeLaw()
{
    bool noThrow = true;
    bool allNull = false;
    try
    {
        allNull = ArtRuntime.Get(null) == null
            && ArtRuntime.Get("") == null
            && ArtRuntime.Get("g10_stem_missing") == null
            && TradeArt.Get(null) == null
            && TradeArt.Get("g10_stem_missing") == null;
    }
    catch (Exception e) { noThrow = false; Lines.Add("FAIL detail - throw: " + e.Message); }
    PassFail("S4.null_safe_lookup", noThrow && allNull,
        "null/empty/missing lookups: null out, zero exceptions");
}

static void Section5FallbackFaceLaw()
{
    bool faces = true;
    try
    {
        faces = TradePlaceholder.KLineBoard() != null
            && TradePlaceholder.EventCard() != null
            && TradePlaceholder.Panel() != null
            && TradePlaceholder.TraderCard("trd_all_in_bro") != null
            && TradePlaceholder.TraderCard("trd_steady_wang") != null
            && TradePlaceholder.Face("g10_any_face", TradePlaceholder.AccentGold, 32) != null;
```

```
            }
            catch (Exception e) { faces = false; Lines.Add("FAIL detail - face throw: " + e.Message); }
            PassFail("S5.procedural_fallback_alive", faces,
                "every wired face renders procedurally while art is absent (X027 iron rule)");
        }

        static void WriteReport()
        {
            var sb = new StringBuilder();
            sb.AppendLine("[ArtCheck] " + (_failed == 0 ? "ALL_GREEN" : "RED")
                + " passed=" + _passed + " failed=" + _failed
                + " at " + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
            foreach (var line in Lines) sb.AppendLine(line);
            string dir = Path.Combine(Application.dataPath, "..", "Logs");
            if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
            File.WriteAllText(Path.Combine(dir, "g10_art_report.txt"), sb.ToString());
            Debug.Log(sb.ToString().Split('\n')[0]);
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Editor\AudioCheck.cs =====
// AudioCheck.cs - the T020 wiring gate (batchmode entry):
// Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyStocks.AudioCheck.Run -logFile <abs>
// Section1: catalog_closed_set_law - the AudioSetup rebuild lands the
//            delivered closed set (14 stems, the G10 audio manifest
//            S1A1/A2/A3 batches) with live clips, unique keys, and
//            nothing extra.
// Section2: stamp_map_law - every wired stamp resolves a catalog stem
//            and the driven opening script fires no silent stamps
//            (every fired stamp is wired, channel-start, or in the
//            documented visual-only family).
// Section3: session_cue_law - the market's own ritual bells resolve
//            across a driven full trading day (open_bell -> opening
//            gong, close_bell -> closing bell).
// Section4: named_cue_law - the six non-stamp face cues resolve stems.
// Section5: null_safe_law - unknown ids and missing stems skip
//            silently (false/null, zero exceptions) on the headless
//            host; the play positive face never throws.
// Section6: hook_family_law - the earworm hook family is wired (open
//            gong + closing bell + the match riff cousin) and the
//            delivered durations sit in the manifest bands.
// Exit codes: 0=all assertions pass  1=fail/exception
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyStocks
```

```
{
    public static class AudioCheck
    {
        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;

        // the delivered closed set (G10 audio manifest, S1A1/A2/A3 batches)
        static readonly string[] ExpectedStems =
        {
            TradeAudio.StemAmbHall, TradeAudio.StemBgmMain, TradeAudio.StemSfxAward,
            TradeAudio.StemSfxCandle, TradeAudio.StemSfxClose, TradeAudio.StemSfxCoin,
            TradeAudio.StemSfxDanmaku, TradeAudio.StemSfxEvent, TradeAudio.StemSfxGap,
            TradeAudio.StemSfxIpo, TradeAudio.StemSfxLimit, TradeAudio.StemSfxMatch,
            TradeAudio.StemSfxOpen, TradeAudio.StemSfxUpgrade,
        };

        // stamps that carry no audio by design (visual / number / narrative faces)
        static readonly HashSet<string> VisualOnlyStamps = new HashSet<string>
        {
            "evt_first_order", "evt_first_fill", "fill_visual", "fill_number",
            "evt_first_coin", "coin_visual", "coin_number",
            "evt_trader_cameo", "evt_first_laugh",
            TradeTierGate.CeremonyAwardVisual, TradeTierGate.CeremonyMcBroadcast,
        };

        public static void Run()
        {
            int code;
            try
            {
                Section1CatalogClosedSetLaw();
                Section2StampMapLaw();
                Section3SessionCueLaw();
                Section4NamedCueLaw();
                Section5NullSafeLaw();
                Section6HookFamilyLaw();
                code = _failed == 0 ? 0 : 1;
            }
            catch (Exception e)
            {
                Lines.Add("[AudioCheck] FAIL exception: " + e);
                _failed++;
                code = 1;
            }
            WriteReport();
            foreach (var line in Lines) Debug.Log(line);
            EditorApplication.Exit(code);
        }
```

```
        static void PassFail(string name, bool ok, string msg)
        {
            if (ok) { _passed++; Lines.Add("PASS " + name + " - " + msg); }
            else { _failed++; Lines.Add("FAIL " + name + " - " + msg); }
        }

        static AudioClip Clip(string stem)
            => AudioRuntime.Catalog != null ? AudioRuntime.Catalog.Get(stem) : null;

        static string SortedJoin(IEnumerable<string> xs)
        {
            var list = new List<string>(xs);
            list.Sort(StringComparer.Ordinal);
            return string.Join(",", list);
        }

        static void Section1CatalogClosedSetLaw()
        {
            int built = AudioSetup.Build();
            var catalog = Resources.Load<AudioCatalog>("AudioCatalog");
            PassFail("S1.catalog_rebuild", catalog != null && built == ExpectedStems.Length,
                "built=" + built + " expected=" + ExpectedStems.Length);

            var keys = new List<string>();
            int liveClips = 0;
            foreach (var e in catalog.entries)
            {
                if (e == null || string.IsNullOrEmpty(e.key)) continue;
                keys.Add(e.key);
                if (e.clip != null) liveClips++;
            }
            var expected = SortedJoin(ExpectedStems);
            var landed = SortedJoin(keys);
            PassFail("S1.closed_set", expected == landed,
                "entries=" + keys.Count + (expected == landed ? " exact-set" : " diff=" + landed));
            PassFail("S1.live_clips", keys.Count == liveClips && keys.Count == ExpectedStems.Length,
                "live=" + liveClips + "/" + keys.Count);

            var distinct = new HashSet<string>(keys, StringComparer.Ordinal);
            PassFail("S1.unique_keys", distinct.Count == keys.Count,
                "distinct=" + distinct.Count + " rows=" + keys.Count);
        }

        static void Section2StampMapLaw()
        {
            int wired = 0, resolved = 0;
            foreach (var kv in TradeAudio.Stamps)
            {
                wired++;
                if (Clip(kv.Value) != null) resolved++;
```

```
            else Lines.Add("FAIL detail - wired stem missing clip: " + kv.Key + " -> " + kv.Value);
        }
        PassFail("S2.wired_stamps_resolve", wired == resolved && wired > 0,
            "wired=" + wired + " resolved=" + resolved);

        // drive the frozen opening script: order in the auction, the
        // bell, the fill, the coin, the event card all fire inside 12s
        var stage = TradeStage.NewLeek();
        stage.Advance(4.0);
        stage.PlaceFirstOrder();
        while (stage.SimTime < 12.0) stage.Advance(0.5);

        int silent = 0;
        foreach (var (id, t) in stage.Stamps)
        {
            if (TradeAudio.StemForStamp(id) != null) continue;
            if (id == TradeAudio.StampTradeHook) continue;            // the channel-start face
            if (VisualOnlyStamps.Contains(id)) continue;              // documented no-audio family
            silent++;
            Lines.Add("FAIL detail - silent stamp: " + id + "@" + t);
        }
        PassFail("S2.no_silent_stamps", silent == 0, "fired=" + stage.Stamps.Count + " silent=" + silent);

        string[] beats = { "evt_open_bell", "fill_sfx", "coin_sfx", "evt_market_event_card", TradeAudio.StampTradeHook };
        int fired = 0;
        foreach (var b in beats) if (stage.TimeOf(b) >= 0) fired++;
        PassFail("S2.wired_beats_fired", fired == beats.Length,
            "fired=" + fired + "/" + beats.Length + " open_bell@" + stage.TimeOf("evt_open_bell").ToString("0.##"));
    }

    static void Section3SessionCueLaw()
    {
        // drive a full trading day until the close bell rings
        var stage = TradeStage.NewLeek();
        stage.Advance(4.0);
        stage.PlaceFirstOrder();
        bool sawOpen = false, sawClose = false;
        int guard = 0;
        while (!sawClose && guard++ < 2000)
        {
            stage.Advance(5.0);
            foreach (var m in stage.Market.SessionMarks)
            {
                if (m.sessionId == "open_bell") sawOpen = true;
                if (m.sessionId == "close_bell") sawClose = true;
            }
        }
        PassFail("S3.bells_observed", sawOpen && sawClose,
            "open=" + sawOpen + " close=" + sawClose + " t=" + stage.SimTime.ToString("0.#") + "s");
```

```
    string openStem = TradeAudio.StemForSession("open_bell");
    string closeStem = TradeAudio.StemForSession("close_bell");
    PassFail("S3.session_stems", openStem == TradeAudio.StemSfxOpen && closeStem == TradeAudio.StemSfxClose
        && Clip(openStem) != null && Clip(closeStem) != null,
        "open->" + openStem + " close->" + closeStem);
}

static void Section4NamedCueLaw()
{
    int wired = 0, resolved = 0;
    foreach (var kv in TradeAudio.Cues)
    {
        wired++;
        if (Clip(kv.Value) != null) resolved++;
        else Lines.Add("FAIL detail - cue stem missing clip: " + kv.Key + " -> " + kv.Value);
    }
    PassFail("S4.named_cues_resolve", wired == resolved && wired == 6,
        "cues=" + wired + " resolved=" + resolved);

    int channels = 0;
    foreach (var ch in TradeAudio.Channels) if (Clip(ch) != null) channels++;
    PassFail("S4.loop_channels_resolve", channels == TradeAudio.Channels.Length,
        "channels=" + channels + "/" + TradeAudio.Channels.Length);
}

static void Section5NullSafeLaw()
{
    PassFail("S5.unknown_lookups_null",
        TradeAudio.StemForSession("bogus_session") == null
        && TradeAudio.StemForStamp("bogus_stamp") == null
        && TradeAudio.StemForCue("bogus_cue") == null
        && AudioRuntime.Get("g10_sfx_missing") == null,
        "unknown ids -> null (no throw)");

    bool noThrow = true;
    bool skipped = false;
    try
    {
        skipped = TradeAudio.Play("g10_sfx_missing") == false
            && TradeAudio.PlayStamp("bogus_stamp") == false
            && TradeAudio.PlayCue("bogus_cue") == false
            && TradeAudio.PlaySession("bogus_session") == false
            && TradeAudio.Play(null) == false;
    }
    catch (Exception e) { noThrow = false; Lines.Add("FAIL detail - throw: " + e.Message); }
    PassFail("S5.missing_skips_silently", noThrow && skipped,
        "missing stems/ids skip false, zero exceptions");

    bool playedFaceNoThrow = true;
    try { TradeAudio.Play(TradeAudio.StemSfxMatch); TradeAudio.StartMainFace(); }
```

```
            catch (Exception e) { playedFaceNoThrow = false; Lines.Add("FAIL detail - play throw: " + e.Message); }
            PassFail("S5.play_positive_no_throw", playedFaceNoThrow,
                "real-stem play + channel start never throw (device face out of gate scope)");
        }

        static void Section6HookFamilyLaw()
        {
            bool open = Clip(TradeAudio.StemSfxOpen) != null;
            bool close = Clip(TradeAudio.StemSfxClose) != null;
            bool match = Clip(TradeAudio.StemSfxMatch) != null;
            PassFail("S6.hook_family_wired", open && close && match,
                "open=" + open + " close=" + close + " match=" + match);

            // manifest duration bands: sfx 1-2s, amb 30s, bgm 60s
            int inBand = 0, checkedSfx = 0;
            foreach (var stem in ExpectedStems)
            {
                var clip = Clip(stem);
                if (clip == null) continue;
                double len = clip.length;
                bool ok;
                if (stem == TradeAudio.StemBgmMain) ok = len >= 50.0 && len <= 70.0;
                else if (stem == TradeAudio.StemAmbHall) ok = len >= 25.0 && len <= 35.0;
                else { checkedSfx++; ok = len >= 0.5 && len <= 3.0; }
                if (ok) inBand++;
                else Lines.Add("FAIL detail - duration out of band: " + stem + " " + len.ToString("0.##") + "s");
            }
            PassFail("S6.duration_bands", inBand == ExpectedStems.Length,
                "in_band=" + inBand + "/" + ExpectedStems.Length + " sfx_checked=" + checkedSfx);
        }

        static void WriteReport()
        {
            var sb = new StringBuilder();
            sb.AppendLine("[AudioCheck] " + (_failed == 0 ? "ALL_GREEN" : "RED")
                + " passed=" + _passed + " failed=" + _failed
                + " at " + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
            foreach (var line in Lines) sb.AppendLine(line);
            string dir = Path.Combine(Application.dataPath, "..", "Logs");
            if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
            File.WriteAllText(Path.Combine(dir, "g10_audio_report.txt"), sb.ToString());
            Debug.Log(sb.ToString().Split('\n')[0]);
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Editor\AudioSetup.cs =====
// AudioSetup.cs - the T020 catalog builder (the X027 ArtSetup paradigm
// applied to audio): scans Assets/_Game/Audio (BGM + SFX subfolders),
// resolves every imported AudioClip by filename stem, and (re)builds
// Assets/_Game/Resources/AudioCatalog.asset. Called by the AudioCheck
```

```
// gate before its assertions (self-healing catalog); also runnable
// standalone: Tuanjie.exe -batchmode -quit -executeMethod Sy.CrazyStocks.AudioSetup.Build
using System.Collections.Generic;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyStocks
{
    public static class AudioSetup
    {
        const string AudioRoot = "Assets/_Game/Audio";
        const string ResourcesFolder = "Assets/_Game/Resources";
        const string CatalogPath = "Assets/_Game/Resources/AudioCatalog.asset";

        /// <summary>(Re)build the audio catalog from the imported clips.
        /// Returns the entry count. Deterministic: entries sort by stem so
        /// two builds land the same key sequence (the parity face).</summary>
        public static int Build()
        {
            var guids = AssetDatabase.FindAssets("t:AudioClip", new[] { AudioRoot });
            var entries = new List<AudioCatalog.Entry>();
            foreach (var guid in guids)
            {
                var path = AssetDatabase.GUIDToAssetPath(guid);
                if (string.IsNullOrEmpty(path)) continue;
                var clip = AssetDatabase.LoadAssetAtPath<AudioClip>(path);
                if (clip == null) continue;
                entries.Add(new AudioCatalog.Entry { key = clip.name, clip = clip });
            }
            entries.Sort((a, b) => string.CompareOrdinal(a.key, b.key));

            if (!AssetDatabase.IsValidFolder(ResourcesFolder))
                AssetDatabase.CreateFolder("Assets/_Game", "Resources");
            var catalog = AssetDatabase.LoadAssetAtPath<AudioCatalog>(CatalogPath);
            if (catalog == null)
            {
                catalog = ScriptableObject.CreateInstance<AudioCatalog>();
                AssetDatabase.CreateAsset(catalog, CatalogPath);
            }
            catalog.entries = entries;
            EditorUtility.SetDirty(catalog);
            AssetDatabase.SaveAssets();
            AssetDatabase.Refresh();
            return entries.Count;
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Editor\BoardCheck.cs =====
// BoardCheck.cs - the boards face gate (G10-T013 acceptance, batchmode
// entry):
```

```
// Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyStocks.BoardCheck.Run -logFile <abs>
// Section1: dragon_board_law - the movers podium: the row sequence
//            re-derives exact (|effective day move| desc, the ordinal id
//            tie-break, the frozen cap), the row domain stays the frozen
//            stock pool, the day carries the market day, and every move
//            recomputes exact against the live market.
// Section2: perf_board_law - the day-return board: the full sequence
//            re-derives exact (exactly the player book plus the frozen
//            crew, ranked desc by the day return with the id tie-break),
//            the player's bp recomputed independent off the open-marked
//            book, every crew book inside its frozen band.
// Section3: opponent_cameo_law - the actor-troupe reconciliation: every
//            crew row's seat and signature key land in the frozen table,
//            the cameo pairing recomputes exact (the stock seed x the
//            day), the same day prints the same crew books twice (pure),
//            the seats vary across days (the board is alive), and the
//            signature lines stay compiled English with zh coverage.
// Section4: revisit_refresh_law - Refresh() is the only writer: the
//            stored board never ticks on its own (a day rollover leaves
//            the stale rows until the revisit), the revisit catches the
//            day up, the same-state refresh prints the same board twice,
//            the appearance telemetry books once per day-stock (the
//            player's lived seal takes the rider seat), and the offline
//            walk lands the boards on the return.
// Section5: card_slot_law - the stage owns the board wired to the market
//            and the SYS07 dragon feed; the showcase card keeps the
//            T011 four-line spec with no board attached and carries the
//            board line as the optional fifth slot when attached (the
//            rank + the top mover recomputed exact).
// Section6: zero_rng_parity_law - two identical scripted runs (visit,
//            teaching fill, crew seal, bell walk, revisit, card
//            compose) fingerprint byte-identical.
// Exit codes: 0=all assertions pass  1=fail/exception
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyStocks
{
    public static class BoardCheck
    {
        const string DayKey = "20991231";

        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;
```

```
public static void Run()
{
    int code;
    try
    {
        Section1DragonBoardLaw();
        Section2PerfBoardLaw();
        Section3OpponentCameoLaw();
        Section4RevisitRefreshLaw();
        Section5CardSlotLaw();
        Section6ZeroRngParityLaw();
        code = _failed == 0 ? 0 : 1;
    }
    catch (Exception e)
    {
        Lines.Add("[BoardCheck] FAIL exception: " + e);
        _failed++;
        code = 1;
    }
    WriteReport();
    foreach (var line in Lines) Debug.Log(line);
    EditorApplication.Exit(code);
}

static void PassFail(string name, bool ok, string msg)
{
    if (ok) { _passed++; Lines.Add("PASS " + name + " - " + msg); }
    else { _failed++; Lines.Add("FAIL " + name + " - " + msg); }
}

static bool HasCjk(string s)
{
    foreach (char ch in s)
        if (ch >= '\u4E00' && ch <= '\u9FFF') return true;
    return false;
}

static string RowsSig(IReadOnlyList<BoardRow> rows)
{
    var sb = new StringBuilder();
    foreach (var r in rows)
        sb.Append(';').Append(r.kind).Append('/').Append(r.day).Append('#').Append(r.rank)
          .Append('@').Append(r.actorId).Append('+').Append(r.stockId).Append('^').Append(r.moveBp);
    return sb.ToString();
}

static List<BoardRow> RowsOf(IReadOnlyList<BoardRow> rows, string kind)
{
    var list = new List<BoardRow>();
```

```
    foreach (var r in rows) if (r.kind == kind) list.Add(r);
    return list;
}

static int AbsBp(int bp) => bp < 0 ? -bp : bp;

/// <summary>The expected dragon podium re-derived independent off
/// the live market: |day move| desc, the ordinal id tie-break,
/// the frozen row cap.</summary>
static List<string> ExpectedDragon(TradeMarket m)
{
    var sorted = new List<StockDef>(TradeData.Stocks());
    sorted.Sort((a, b) =>
    {
        int absA = AbsBp(m.EffectiveDayBp(m.Stock(a.id)));
        int absB = AbsBp(m.EffectiveDayBp(m.Stock(b.id)));
        if (absA != absB) return absB - absA;
        return string.CompareOrdinal(a.id, b.id);
    });
    var seq = new List<string>();
    int cap = Math.Min(TradeBoard.DragonRowsMax, sorted.Count);
    for (int i = 0; i < cap; i++) seq.Add(sorted[i].id);
    return seq;
}

/// <summary>The expected perf sequence re-derived independent:
/// the player's day bp off the open-marked book plus the crew's
/// pure seed books, ranked desc with the id tie-break.</summary>
static List<(string id, int bp)> ExpectedPerf(TradeMarket m, int playerBp)
{
    var books = new List<(string, int)> { (TradeBoard.ActorYou, playerBp) };
    foreach (var o in TradeData.BoardOpponents())
        books.Add((o.id, TradeData.OpponentDayBp(o, m.Day)));
    books.Sort((x, y) =>
    {
        if (x.Item2 != y.Item2) return y.Item2 - x.Item2;
        return string.CompareOrdinal(x.Item1, y.Item1);
    });
    return books;
}

static int PlayerDayBpIndependent(TradeMarket m)
{
    int openBook = m.WalletHc;
    foreach (var def in TradeData.Stocks())
    {
        var s = m.Stock(def.id);
        if (s != null) openBook += s.shares * s.openHc;
    }
    if (openBook <= 0) return 0;
```

```
    return (int)(((long)m.TotalAssetsHc - openBook) * 10000 / openBook);
}

/// <summary>Probes a gang-up seal target among the current podium
/// movers (the appearance face must land inside the podium): the
/// first podium stock whose day state plus up to three max-band
/// pre-pumps lands inside the seal window (one push away from the
/// board). Applies the pre-pumps and returns the target id, or
/// null when nothing fits (an honest red, never a fabricated
/// pass).</summary>
static string ProbeSealTarget(TradeMarket m)
{
    foreach (var id in ExpectedDragon(m))
    {
        var s = m.Stock(id);
        int limitBp = s.board.limitPct * 100;
        for (int k = 0; k <= 3; k++)
        {
            int bp = m.EffectiveDayBp(s) + k * TradeData.EventDisturbMaxPct * 100;
            if (bp < limitBp && bp + TradeData.EventDisturbMaxPct * 100 >= limitBp)
            {
                for (int i = 0; i < k; i++) m.ApplyEventDisturbance(id, TradeData.EventDisturbMaxPct);
                return id;
            }
        }
    }
    return null;
}

static void Section1DragonBoardLaw()
{
    var m = new TradeMarket(0);
    var board = new TradeBoard(m, null);
    bool emptyBefore = board.Rows.Count == 0;   // the boards never tick on their own
    board.Refresh();
    var dragon = RowsOf(board.Rows, TradeBoard.KindDragon);
    var expected = ExpectedDragon(m);
    bool sequenceExact = dragon.Count == expected.Count;
    for (int i = 0; i < Math.Min(dragon.Count, expected.Count); i++)
        if (dragon[i].stockId != expected[i]) sequenceExact = false;
    bool podium = dragon.Count == Math.Min(TradeBoard.DragonRowsMax, TradeData.Stocks().Length);
    bool domain = true;
    var pool = new HashSet<string>();
    foreach (var def in TradeData.Stocks()) pool.Add(def.id);
    foreach (var r in dragon) if (!pool.Contains(r.stockId)) domain = false;
    bool movesExact = true;
    foreach (var r in dragon)
        if (r.moveBp != m.EffectiveDayBp(m.Stock(r.stockId))) movesExact = false;
    bool dayCarried = true;
    foreach (var r in dragon) if (r.day != m.Day) dayCarried = false;
```

```
            bool ranksSequenced = true;
            for (int i = 0; i < dragon.Count; i++) if (dragon[i].rank != i + 1) ranksSequenced = false;
            PassFail("dragon_board_law",
                emptyBefore && sequenceExact && podium && domain && movesExact && dayCarried && ranksSequenced,
                "subverdicts emptyBefore=" + emptyBefore + " sequenceExact=" + sequenceExact
                + " podium=" + podium + " [" + dragon.Count + " rows: " + string.Join(",", expected) + "]"
                + " domain=" + domain + " movesExact=" + movesExact + " dayCarried=" + dayCarried + " ranks=" + ranksSequenced
                + " - the movers podium re-derives exact (|effective day move| desc, the ordinal tie-break, the frozen cap), the row
domain stays the frozen stock pool, the day carries the market day and every move recomputes exact against the live market");
        }

        static void Section2PerfBoardLaw()
        {
            var stage = TradeStage.NewLeek();
            stage.Advance(4.0);
            stage.PlaceFirstOrder();
            stage.Advance(80.0);   // the open bell fills the teaching order (a live book to mark)
            var board = stage.Board;
            board.Refresh();
            var perf = RowsOf(board.Rows, TradeBoard.KindPerf);
            int expectBp = PlayerDayBpIndependent(stage.Market);
            var expected = ExpectedPerf(stage.Market, expectBp);
            bool sequenceExact = perf.Count == expected.Count;
            for (int i = 0; i < Math.Min(perf.Count, expected.Count); i++)
                if (perf[i].actorId != expected[i].id || perf[i].moveBp != expected[i].bp) sequenceExact = false;
            bool census = perf.Count == 1 + TradeData.BoardOpponentCount;
            int playerRows = 0;
            foreach (var r in perf) if (r.actorId == TradeBoard.ActorYou) playerRows++;
            bool onePlayer = playerRows == 1;
            bool playerExact = board.PlayerDayBp() == expectBp;
            bool inBand = true;
            foreach (var r in perf)
            {
                var o = FindOpp(r.actorId);
                if (o == null) continue;
                if (r.moveBp < -o.bandBp || r.moveBp > o.bandBp) inBand = false;   // inside the frozen band
                if (r.moveBp != TradeData.OpponentDayBp(o, stage.Market.Day)) inBand = false;   // the pure seed derivation
            }
            bool stockBlank = true;
            foreach (var r in perf) if (r.stockId != "") stockBlank = false;
            PassFail("perf_board_law",
                sequenceExact && census && onePlayer && playerExact && inBand && stockBlank,
                "subverdicts sequenceExact=" + sequenceExact + " census=" + census + " [" + perf.Count + " rows]"
                + " onePlayer=" + onePlayer + " playerExact=" + playerExact
                + " [player " + board.PlayerDayBp() + "bp vs expect " + expectBp + "bp]"
                + " inBand=" + inBand + " stockBlank=" + stockBlank
                + " - the day-return board re-derives exact (exactly the player book plus the frozen crew, ranked desc with the id
tie-break), the player's bp recomputes independent off the open-marked book, and every crew book stays inside its frozen band as the
pure seed derivation");
        }

        static OpponentDef FindOpp(string id)
        {
```

```
    foreach (var o in TradeData.BoardOpponents()) if (o.id == id) return o;
    return null;
}

static void Section3OpponentCameoLaw()
{
    var m = new TradeMarket(0);
    var board = new TradeBoard(m, null);
    board.Refresh();
    var dragon = RowsOf(board.Rows, TradeBoard.KindDragon);
    bool tableSealed = TradeData.BoardOpponentCount == 4
        && TradeData.BoardOpponents().Length == TradeData.BoardOpponentCount;
    var seen = new HashSet<string>();
    foreach (var o in TradeData.BoardOpponents()) if (!seen.Add(o.id)) tableSealed = false;
    bool pairingExact = true;
    foreach (var r in dragon)
    {
        if (r.actorId == TradeBoard.ActorYou) continue;
        var def = StockDefOf(r.stockId);
        var expect = TradeBoard.CameoFor(def, m.Day);
        if (r.actorId != expect.id || r.nameKey != expect.nameKey) pairingExact = false;
    }
    bool cameoDomain = true;
    foreach (var r in dragon)
        if (r.actorId != TradeBoard.ActorYou && FindOpp(r.actorId) == null) cameoDomain = false;
    bool pure = true;
    bool alive = true;
    foreach (var o in TradeData.BoardOpponents())
    {
        if (TradeData.OpponentDayBp(o, 5) != TradeData.OpponentDayBp(o, 5)) pure = false;
        var vals = new HashSet<int>();
        for (int day = 1; day <= 10; day++) vals.Add(TradeData.OpponentDayBp(o, day));
        if (vals.Count < 2) alive = false;   // the seat's book moves day to day (never a frozen face)
    }
    bool copy = true;
    foreach (var o in TradeData.BoardOpponents())
    {
        string en = TradeTexts.EnText(o.nameKey);
        if (en == o.nameKey || HasCjk(en) || !en.Contains(" - ")) copy = false;   // the signature one-liner format
    }
    string zhPath = Path.Combine(Application.dataPath, "_Game", "Configs", "Texts_zhCN.json");
    var zh = File.Exists(zhPath) ? TradeTexts.ParseZh(File.ReadAllText(zhPath)) : new Dictionary<string, string>();
    bool zhCover = zh.Count == TradeTexts.En.Count;
    foreach (var o in TradeData.BoardOpponents())
        if (!zh.TryGetValue(o.nameKey, out var v) || string.IsNullOrEmpty(v)) zhCover = false;
    bool boardCopy = !HasCjk(TradeTexts.EnText(TradeTexts.KeyBoardTitle))
        && !HasCjk(TradeTexts.EnText(TradeTexts.KeyBoardDragon))
        && !HasCjk(TradeTexts.EnText(TradeTexts.KeyBoardPerf))
        && !HasCjk(TradeTexts.EnText(TradeTexts.KeyBoardYou))
        && !HasCjk(TradeTexts.EnText(TradeTexts.KeyBoardCardLine));
```

```
            PassFail("opponent_cameo_law",
                tableSealed && pairingExact && cameoDomain && pure && alive && copy && zhCover && boardCopy,
                "subverdicts tableSealed=" + tableSealed + " pairingExact=" + pairingExact
                + " cameoDomain=" + cameoDomain + " pure=" + pure + " alive=" + alive
                + " copy=" + copy + " zhCover=" + zhCover + " [" + zh.Count + "/" + TradeTexts.En.Count + " keys] boardCopy=" +
boardCopy
                + " - the crew table freezes four unique seats, every mover row's cameo recomputes exact (the stock seed x the day),
the same day prints the same crew books twice while the seats stay alive across days, and the signature one-liners stay compiled
English with full zh coverage");
        }

        static StockDef StockDefOf(string id)
        {
            foreach (var d in TradeData.Stocks()) if (d.id == id) return d;
            return null;
        }

        static void Section4RevisitRefreshLaw()
        {
            var stage = TradeStage.NewLeek();
            var board = stage.Board;
            stage.Advance(4.0);
            stage.PlaceFirstOrder();
            stage.Advance(80.0);
            board.Refresh();
            int day1 = stage.Market.Day;
            string staleSig = RowsSig(board.Rows);
            stage.Advance(1800.0);   // cross the close bell + the rollover (day 2)
            int day2 = stage.Market.Day;
            bool noSelfTick = day2 != day1 && RowsSig(board.Rows) == staleSig;   // the stored board stays stale until the revisit
            board.Refresh();
            bool caughtUp = true;
            foreach (var r in board.Rows) if (r.day != day2) caughtUp = false;
            string visit2Sig = RowsSig(board.Rows);
            board.Refresh();
            bool idempotent = RowsSig(board.Rows) == visit2Sig;   // same-state refresh prints the same board
            // the appearance face: hold a sealing podium stock -> the rider seat + once-per-day telemetry
            var m = stage.Market;
            string sealTarget = ProbeSealTarget(m);
            bool sealPath = sealTarget != null;
            if (sealPath)
            {
                var s = m.Stock(sealTarget);
                int shares = (m.WalletHc / 2) / s.priceHc;   // half the wallet: the fee always fits
                m.PlaceOrder(sealTarget, true, true, 0, shares);   // the live book holds the mover
                m.ApplyEventDisturbance(sealTarget, TradeData.EventDisturbMaxPct);   // one push over the line
            }
            board.Refresh();
            bool seated = false;
            if (sealPath)
                foreach (var r in board.Rows)
                    if (r.kind == TradeBoard.KindDragon && r.stockId == sealTarget && r.actorId == TradeBoard.ActorYou) seated =
true;
            string faceLine = "evt_dragon_face@" + m.Day + ":" + sealTarget;
```

```
            int faceCount = 0;
            foreach (var f in board.Funnel) if (f == faceLine) faceCount++;
            board.Refresh();   // the same-day revisit never double-books
            int faceCount2 = 0;
            foreach (var f in board.Funnel) if (f == faceLine) faceCount2++;
            bool faceOnce = seated && faceCount == 1 && faceCount2 == 1;
            // the offline walk: the boards land on the return visit
            var off = new TradeOffline(stage, null, null, () => DayKey);
            off.Settle(2 * 1800.0 + 400.0);   // about two game days away
            int walkedDay = stage.Market.Day;
            board.Refresh();
            bool returnCaught = true;
            foreach (var r in board.Rows) if (r.day != walkedDay) returnCaught = false;
            PassFail("revisit_refresh_law",
                noSelfTick && caughtUp && idempotent && sealPath && faceOnce && returnCaught,
                "subverdicts noSelfTick=" + noSelfTick + " [d" + day1 + "->d" + day2 + " stale held]"
                + " caughtUp=" + caughtUp + " idempotent=" + idempotent
                + " sealPath=" + sealPath + " [target=" + sealTarget + "] seated=" + seated
                + " faceOnce=" + faceOnce + " [" + faceCount2 + " bookings] returnCaught=" + returnCaught + " [walked to d" +
walkedDay + "]"
                + " - Refresh() is the only writer: a day rollover leaves the stale board until the revisit catches it up, the same-
state refresh prints the same board twice, the player's lived seal takes the rider seat with the appearance telemetry booked once per
day-stock, and the offline walk lands the boards on the return visit");
        }

        static void Section5CardSlotLaw()
        {
            var stage = TradeStage.NewLeek();
            var probe = stage.Board;
            bool owned = probe != null && probe.Market == stage.Market && probe.IpoFund == stage.IpoFund;
            var sc = new TradeShowcase(stage);
            var bare = sc.ComposeCard();
            bool fourLine = bare != null && bare.Lines.Length == 4;   // the T011 spec untouched with no board attached
            sc.Board = stage.Board;
            stage.Board.Refresh();
            var card = sc.ComposeCard();
            bool fiveLine = card != null && card.Lines.Length == 5;
            // the slot line recomputed independent: the rank + the total + the top mover
            var m = stage.Market;
            int playerBp = PlayerDayBpIndependent(m);
            var perf = ExpectedPerf(m, playerBp);
            int rank = 1;
            for (int i = 0; i < perf.Count; i++)
                if (perf[i].id == TradeBoard.ActorYou) { rank = i + 1; break; }   // the player's seat in the re-derived ranking
            string topMover = "";
            int topBp = 0;
            foreach (var id in ExpectedDragon(m))
            {
                topMover = id;
                topBp = m.EffectiveDayBp(m.Stock(id));
                break;
            }
            string expectLine = string.Format(CultureInfo.InvariantCulture,
```

```
                TradeTexts.EnText(TradeTexts.KeyBoardCardLine), rank, perf.Count, topMover,
                (topBp >= 0 ? "+" : "") + (topBp / 100.0).ToString("0.0", CultureInfo.InvariantCulture) + "%");
            bool slotExact = fiveLine && card.Lines[4] == expectLine && !HasCjk(string.Join("|", card.Lines));
            bool endingNull = sc.ComposeEndingCard() == null;   // pre-summit law holds with the board attached
            PassFail("card_slot_law",
                owned && fourLine && fiveLine && slotExact && endingNull,
                "subverdicts owned=" + owned + " fourLine=" + fourLine + " fiveLine=" + fiveLine
                + " slotExact=" + slotExact + " [rank #" + rank + "/" + perf.Count + " top " + topMover + " " + topBp + "bp]
endingNull=" + endingNull
                + " - the stage owns the board wired to the market + the SYS07 feed, the showcase card keeps its four-line spec bare
and carries the board line as the optional fifth slot when attached (the rank and the top mover recomputed exact), and the ending
card stays null before the summit");
        }

        static string ScriptedBoardRun()
        {
            var stage = TradeStage.NewLeek();
            var board = stage.Board;
            stage.Advance(4.0);
            stage.PlaceFirstOrder();
            stage.Advance(80.0);   // the open bell fills the teaching order
            board.Refresh();   // the first visit
            // the crew seal (the SYS07 dragon feed): a rich T3 joins the push
            var gate = stage.Gate;
            long rich = TradeData.Tiers()[4].assetGate * 100;
            gate.RaiseFlag(TradeData.Tiers()[1].flagEventId);
            gate.Tick(rich);
            gate.RaiseFlag(TradeData.Tiers()[2].flagEventId);
            gate.Tick(rich);
            string target = ProbeSealTarget(stage.Market);
            if (target == null) return "PROBE_FAIL";
            stage.IpoFund.GangUp(target);
            board.Refresh();   // the revisit lands the player on the dragon row
            while (stage.SimTime < 3700.0) stage.Advance(30.0);   // across the close bell + the rollover
            board.Refresh();   // the return visit catches the new day up
            var sc = new TradeShowcase(stage) { Board = board };
            var card = sc.ComposeCard();
            return board.Fingerprint() + "||card" + card.Lines.Length + "||" + stage.OpeningSig();
        }

        static void Section6ZeroRngParityLaw()
        {
            string a = ScriptedBoardRun();
            string b = ScriptedBoardRun();
            PassFail("zero_rng_parity_law",
                a == b && a.Length > 0 && !a.StartsWith("PROBE_FAIL"),
                "two identical scripted runs (visit, teaching fill, crew seal, bell walk, revisit, card compose) fingerprint byte-
identical ("
                + (a == b ? a.Length : 0) + " chars, zero RNG anywhere in the stream)");
        }

        static void WriteReport()
        {
            var sb = new StringBuilder();
```

```
            sb.AppendLine("[BoardCheck] " + (_failed == 0 ? "ALL_GREEN" : "RED")
                + " passed=" + _passed + " failed=" + _failed
                + " at " + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
            foreach (var line in Lines) sb.AppendLine(line);
            string dir = Path.Combine(Application.dataPath, "..", "Logs");
            if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
            File.WriteAllText(Path.Combine(dir, "g10_board_report.txt"), sb.ToString());
            Debug.Log(sb.ToString().Split('\n')[0]);
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Editor\BootSequenceCheck.cs =====
using System.IO;
using System.Text;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;

namespace Sy.CrazyStocks
{
    /// <summary>
    /// Seven-step boot dry-run (batchmode entry):
    /// Tuanjie.exe -batchmode -quit -executeMethod Sy.CrazyStocks.BootSequenceCheck.Run -logFile <abs>
    /// Exit codes: 0=all steps pass and well formed  1=fail/exception  3=scenes missing (run ProjectSetup first)
    /// </summary>
    public static class BootSequenceCheck
    {
        public static void Run()
        {
            int code;
            try
            {
                string mainAsset = "Assets/_Game/Scenes/S10_Main.unity";
                string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
                if (!File.Exists(Path.Combine(projectRoot, mainAsset)))
                {
                    Debug.LogError("[BootSequenceCheck] main scene missing - run ProjectSetup first");
                    EditorApplication.Exit(3);
                    return;
                }

                Boot.BootServices.RegisterAll();
                bool switched = false;
                var sequence = new Boot.BootSequence(delegate
                {
                    EditorSceneManager.OpenScene(mainAsset, OpenSceneMode.Single);
                    switched = true;
                    return true;
                });
                bool allOk = sequence.RunAll();
```

```
                var sb = new StringBuilder();
                foreach (var line in sequence.Lines) sb.AppendLine(line);
                var logsDir = Path.Combine(projectRoot, "Logs");
                if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
                File.WriteAllText(Path.Combine(logsDir, "boot_sequence_report.txt"), sb.ToString(), new UTF8Encoding(false));

                bool wellFormed = sequence.Lines.Count == Boot.BootSequence.TotalSteps && switched;
                for (int i = 0; i < sequence.Lines.Count; i++)
                {
                    Debug.Log(sequence.Lines[i]);
                    if (!sequence.Lines[i].StartsWith("[BOOT] " + (i + 1) + "/" + Boot.BootSequence.TotalSteps + " "))
                        wellFormed = false;
                }

                code = allOk && wellFormed ? 0 : 1;
            }
            catch (System.Exception e)
            {
                Debug.LogError("[BootSequenceCheck] exception: " + e);
                code = 1;
            }
            EditorApplication.Exit(code);
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Editor\BuildService.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyStocks
{
    /// <summary>
    /// Compile zero-warning gate stub (batchmode entry).
    /// Usage: Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyStocks.BuildService.CompileCheck -logFile <abs>
    /// Exit codes: 0=zero error zero warning  1=has error  2=has warning  3=no -logFile  4=exception
    /// </summary>
    public static class BuildService
    {
        public const int Ok = 0;
        public const int HasError = 1;
        public const int HasWarning = 2;
        public const int NoLogFile = 3;
        public const int Exception = 4;

        public static void CompileCheck()
```

```
{
    var report = new CompileCheckReport
    {
        project = "CrazyStocks",
        unityVersion = Application.unityVersion,
        atUtc = DateTime.UtcNow.ToString("o"),
        errors = new List<string>(),
        warnings = new List<string>(),
    };

    int code = Ok;
    try
    {
        string logPath = FindArgValue("-logFile");
        if (string.IsNullOrEmpty(logPath) || !File.Exists(logPath))
        {
            report.status = "no_log_file";
            code = NoLogFile;
        }
        else
        {
            ScanLog(logPath, report);
            if (report.errors.Count > 0) { report.status = "error"; code = HasError; }
            else if (report.warnings.Count > 0) { report.status = "warning"; code = HasWarning; }
            else { report.status = "ok"; code = Ok; }
        }
    }
    catch (Exception e)
    {
        report.status = "exception";
        report.exception = e.Message + "\n" + e.StackTrace;
        code = Exception;
    }

    WriteReport(report);
    EditorApplication.Exit(code);
}

/// <summary>Reads a command line value, supporting both "-logFile x" and "-logFile=x".</summary>
internal static string FindArgValue(string name)
{
    var args = Environment.GetCommandLineArgs();
    for (int i = 0; i < args.Length; i++)
    {
        if (string.Equals(args[i], name, StringComparison.OrdinalIgnoreCase))
        {
            if (i + 1 < args.Length) return args[i + 1];
        }
        else if (args[i].StartsWith(name + "=", StringComparison.OrdinalIgnoreCase))
        {
```

```
                return args[i].Substring(name.Length + 1);
            }
        }
        return null;
    }

    /// <summary>Counts compile errors/warnings (in batchmode compilation happens before executeMethod).</summary>
    static void ScanLog(string logPath, CompileCheckReport report)
    {
        using var fs = new FileStream(logPath, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
        using var sr = new StreamReader(fs, Encoding.UTF8);
        var seenErrors = new HashSet<string>();
        var seenWarnings = new HashSet<string>();
        string line;
        while ((line = sr.ReadLine()) != null)
        {
            if (line.IndexOf(": error CS", StringComparison.Ordinal) >= 0
                || line.IndexOf("Scripts have compiler errors", StringComparison.Ordinal) >= 0
                || line.IndexOf("Error building Player", StringComparison.Ordinal) >= 0)
            {
                if (seenErrors.Add(Trim(line))) report.errors.Add(Trim(line));
            }
            else if (line.IndexOf(": warning CS", StringComparison.Ordinal) >= 0)
            {
                if (seenWarnings.Add(Trim(line))) report.warnings.Add(Trim(line));
            }
        }
        report.errorCount = report.errors.Count;
        report.warningCount = report.warnings.Count;
    }

    static string Trim(string s)
    {
        s = s.TrimEnd();
        return s.Length > 500 ? s.Substring(0, 500) + "..." : s;
    }

    static void WriteReport(CompileCheckReport report)
    {
        try
        {
            var projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
            var logsDir = Path.Combine(projectRoot, "Logs");
            if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
            File.WriteAllText(
                Path.Combine(logsDir, "compile_report.json"),
                JsonUtility.ToJson(report, true),
                new UTF8Encoding(false));
            Console.WriteLine($"[CompileCheck] status={report.status} errors={report.errorCount} warnings={report.warningCount} -
> Logs/compile_report.json");
        }
```

```
            catch (Exception e)
            {
                Console.WriteLine("[CompileCheck] report write failed: " + e.Message);
            }
        }

        /// <summary>
        /// T012 build face: WeChat minigame package build (batchmode).
        /// Ported from Sy.G02.BuildService.BuildWeixin (X123 lineage): same
        /// raw-container format pin (X123 law - no public API face for the
        /// weixinMiniGame* keys, drive the serialized key, restore in
        /// finally) and same SY_WECHAT bridge define (com.sy.platform
        /// WxPlatformSuite only compiles under it). G10 has no grey-cohort
        /// arm, so no SY_SUBMISSION define is injected.
        /// Exit 0 = build succeeded, 1 = switch/build failed, 4 = exception.
        /// Usage: Tuanjie.exe -batchmode -quit -projectPath &lt;p&gt; -executeMethod Sy.CrazyStocks.BuildService.BuildWeixin -
logFile &lt;abs&gt;
        /// </summary>
        public static void BuildWeixin()
        {
            int code;
            SerializedObject psObj = null;
            int fmtPrev = 0;
            bool pinned = false;
            try
            {
                var scenes = EditorBuildSettings.scenes;
                if (scenes == null || scenes.Length == 0)
                {
                    Console.WriteLine("[BuildWeixin] no scenes pinned in EditorBuildSettings");
                    EditorApplication.Exit(1);
                    return;
                }
                var levelList = new List<string>();
                foreach (var s in scenes)
                    if (s.enabled) levelList.Add(s.path);
                if (levelList.Count == 0)
                {
                    Console.WriteLine("[BuildWeixin] no enabled scenes");
                    EditorApplication.Exit(1);
                    return;
                }

                bool switched = EditorUserBuildSettings.SwitchActiveBuildTarget(
                    BuildTargetGroup.WeixinMiniGame, BuildTarget.WeixinMiniGame);
                Console.WriteLine("[BuildWeixin] switchActiveTarget=" + switched
                    + " scenes=" + levelList.Count);
                if (!switched) { EditorApplication.Exit(1); return; }

                string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
                string outDir = Path.Combine(projectRoot, "MiniGame", "weixin", "build");
```

```
Directory.CreateDirectory(outDir);

// X123: pin the raw-container format before building (brotli
// output would break the later 4MB main-package split contract).
var psAll = UnityEngine.Resources.FindObjectsOfTypeAll(typeof(PlayerSettings));
if (psAll == null || psAll.Length == 0)
{
    Console.WriteLine("[BuildWeixin] X123 abort: PlayerSettings instance not found");
    EditorApplication.Exit(1);
    return;
}
psObj = new SerializedObject(psAll[0]);
var fmtProp = psObj.FindProperty("weixinMiniGameCompressionFormat");
if (fmtProp == null)
{
    Console.WriteLine("[BuildWeixin] X123 abort: serialized key weixinMiniGameCompressionFormat missing");
    EditorApplication.Exit(1);
    return;
}
fmtPrev = fmtProp.intValue;
fmtProp.intValue = 0;
psObj.ApplyModifiedProperties();
Console.WriteLine("[BuildWeixin] X123 fmtPin prev=" + fmtPrev
    + " set=0 readback=" + fmtProp.intValue);
if (fmtProp.intValue != 0)
{
    Console.WriteLine("[BuildWeixin] X123 abort: pin readback mismatch - no build");
    EditorApplication.Exit(1);
    return;
}
pinned = true;

string[] buildDefines = { "SY_WECHAT" };

var report = BuildPipeline.BuildPlayer(new BuildPlayerOptions
{
    scenes = levelList.ToArray(),
    locationPathName = outDir,
    target = BuildTarget.WeixinMiniGame,
    options = BuildOptions.None,
    extraScriptingDefines = buildDefines,
});

string result = report.summary.result.ToString();
long totalBytes = (long)report.summary.totalSize;
Console.WriteLine("[BuildWeixin] result=" + result
    + " totalSizeBytes=" + totalBytes
    + " errors=" + report.summary.totalErrors
    + " warnings=" + report.summary.totalWarnings);
```

```
            try
            {
                var logsDir = Path.Combine(projectRoot, "Logs");
                if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
                File.WriteAllText(
                    Path.Combine(logsDir, "build_weixin_report.json"),
                    "{\"result\":\"" + result + "\",\"totalSizeBytes\":" + totalBytes
                    + ",\"errors\":" + report.summary.totalErrors
                    + ",\"warnings\":" + report.summary.totalWarnings
                    + ",\"defineInjected\":\"" + string.Join(";", buildDefines) + "\""
                    + ",\"atUtc\":\"" + DateTime.UtcNow.ToString("o") + "\"}",
                    new UTF8Encoding(false));
            }
            catch (Exception e) { Console.WriteLine("[BuildWeixin] report write failed: " + e.Message); }

            code = report.summary.result == UnityEditor.Build.Reporting.BuildResult.Succeeded ? 0 : 1;
        }
        catch (Exception e)
        {
            Console.WriteLine("[BuildWeixin] exception: " + e.Message + " | " + e.StackTrace);
            code = 1;
        }
        finally
        {
            if (pinned && psObj != null)
            {
                try
                {
                    psObj.Update();
                    var rp = psObj.FindProperty("weixinMiniGameCompressionFormat");
                    if (rp != null)
                    {
                        rp.intValue = fmtPrev;
                        psObj.ApplyModifiedProperties();
                        Console.WriteLine("[BuildWeixin] X123 fmtRestored=" + rp.intValue);
                    }
                    else
                    {
                        Console.WriteLine("[BuildWeixin] X123 WARNING: restore key missing");
                    }
                }
                catch (Exception rex)
                {
                    Console.WriteLine("[BuildWeixin] X123 restore failed: " + rex.Message);
                }
            }
        }
        EditorApplication.Exit(code);
    }
}
```

```
    [Serializable]
    public class CompileCheckReport
    {
        public string project;
        public string unityVersion;
        public string atUtc;
        public string status;
        public int errorCount;
        public int warningCount;
        public List<string> errors;
        public List<string> warnings;
        public string exception;
    }
}
// ===== FILE: Assets\_Game\Scripts\Editor\ContentCheck.cs =====
// ContentCheck.cs - the S1 content-debt trio gate (G10-T015 acceptance,
// batchmode entry):
// Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyStocks.ContentCheck.Run -logFile <abs>
// Section1: sadness_negative_correlation_law - the three counter-cyclical
//            assets (M16): the closed set holds its ids and name keys on
//            both locale faces, the strong-probe yields move strictly
//            against the index, a flat tape still wobbles (never a
//            risk-free hedge), the stage-wired close bell settles every
//            day with the exact re-derivable law, and the noodle
//            position grows on the strong red days (the hedge works).
// Section2: intel_true_false_half_law - the three guru schools (M17):
//            the closed set, the fee table, the accuracy band parked
//            around one half (never pure truth, never pure fiction),
//            the zero-RNG truth flag, the empirical half-rate over 60
//            days, the trust wiring (a false call fires the comedy scam
//            event - never a game over), the degree fires exactly once
//            at the third scam, and a true call books the hit with the
//            direction matching the actual print.
// Section3: replay_template_render_law - the daily theater (M23): the
//            18-key pool holds on both locale faces with zero CJK in
//            the compiled English, the template/cast rotation is exact
//            by day, the LIVE close books one formal replay per bell
//            with the exact re-derived slot substitution, a REPLAY
//            session books the skip line and never a formal compose,
//            and the whole stream fingerprints byte-identical on a
//            double run (zero RNG).
// Exit codes: 0=all assertions pass  1=fail/exception
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;
```

```
namespace Sy.CrazyStocks
{
    public static class ContentCheck
    {
        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;

        public static void Run()
        {
            int code;
            try
            {
                Section1SadnessLaw();
                Section2IntelLaw();
                Section3ReplayLaw();
                Lines.Add("RESULT=" + (_failed == 0 ? "ALL_GREEN" : "FAIL") + " passed=" + _passed + " failed=" + _failed);
                code = _failed == 0 ? 0 : 1;
            }
            catch (System.Exception e)
            {
                Lines.Add("[ContentCheck] FAIL exception: " + e);
                _failed++;
                code = 1;
            }
            WriteReport();
            EditorApplication.Exit(code);
        }

        static void Assert(string name, bool cond, string msg)
        {
            if (cond) _passed++; else _failed++;
            Lines.Add("[ContentCheck] " + name + (cond ? " PASS - " : " FAIL - ") + msg);
        }

        static void WriteReport()
        {
            try
            {
                string logs = Path.Combine(Application.dataPath, "..", "Logs");
                Directory.CreateDirectory(logs);
                File.WriteAllText(Path.Combine(logs, "g10_content_report.txt"),
                    string.Join("\n", Lines.ToArray()) + "\n", new UTF8Encoding(false));
            }
            catch (System.Exception e)
            {
                Debug.LogError("[ContentCheck] report write failed: " + e);
            }
        }
```

```
// ---- helpers ----
static Dictionary<string, string> LoadZh()
{
    string path = Path.Combine(Application.dataPath, "_Game", "Configs", "Texts_zhCN.json");
    return File.Exists(path) ? TradeTexts.ParseZh(File.ReadAllText(path)) : new Dictionary<string, string>();
}

static int CloseBellCount(TradeMarket m)
{
    int n = 0;
    for (int i = 0; i < m.SessionMarks.Count; i++)
        if (m.SessionMarks[i].sessionId == "close_bell") n++;
    return n;
}

/// <summary>The day counter of the most recent close bell (the
/// walk stays inside the post-bell interlude when it reads).</summary>
static int LastBellDay(TradeMarket m)
{
    for (int i = m.SessionMarks.Count - 1; i >= 0; i--)
        if (m.SessionMarks[i].sessionId == "close_bell") return m.SessionMarks[i].day;
    return -1;
}

static bool HasCjk(string s)
{
    foreach (char ch in s)
        if ((ch >= '\u4E00' && ch <= '\u9FFF') || (ch >= '\u3000' && ch <= '\u303F') || (ch >= '\uFF00' && ch <= '\uFFEF'))
            return true;
    return false;
}

// ---- Section 1: sadness_negative_correlation_law ----
static void Section1SadnessLaw()
{
    var zh = LoadZh();
    var assets = TradeContent.SadAssets;
    bool closed = assets.Count == TradeContent.SadnessAssetCount
        && assets[0].id == TradeContent.SadNoodleId
        && assets[1].id == TradeContent.SadCafeId
        && assets[2].id == TradeContent.SadFireworksId;
    bool names = true;
    foreach (var a in assets)
        if (TradeTexts.EnText(a.nameKey).Length <= a.nameKey.Length
            || !zh.ContainsKey(a.nameKey) || string.IsNullOrEmpty(zh[a.nameKey])) names = false;
    bool betas = assets[0].betaPermille == TradeContent.SadNoodleBetaPermille
        && assets[1].betaPermille == TradeContent.SadCafeBetaPermille
        && assets[2].betaPermille == TradeContent.SadFireworksBetaPermille;

    // the strong probes: the yield moves strictly against the tape
```

```
// (the hedge leg outruns the own-wave band on every probe)
int[] probes = { 500, -500, 700, -700 };
bool opposition = true;
foreach (var a in assets)
    foreach (int p in probes)
        if (Math.Sign(TradeContent.SadnessYieldBp(a, p, 3)) != -Math.Sign(p)) opposition = false;

// the flat tape still wobbles: never a fixed risk-free print
var flatYields = new HashSet<int>();
bool flatNonZero = false;
var noodle = TradeContent.SadAsset(TradeContent.SadNoodleId);
for (int d = 1; d <= 8; d++)
{
    int y = TradeContent.SadnessYieldBp(noodle, 0, d);
    flatYields.Add(y);
    if (y != 0) flatNonZero = true;
}
bool wobble = flatYields.Count > 1 && flatNonZero;

// the stage-wired walk: every close bell settles all three
// assets with the exact re-derivable law. The walk steps
// 30s at a time and reads inside the post-bell interlude
// (the day print is final there, the rollover has not hit).
var st = TradeStage.NewLeek();
st.Advance(0.5);
st.PlaceFirstOrder();
st.Content.Invest(TradeContent.SadNoodleId, 100000);
int days = 30;
long valueBefore = st.Content.SadnessValueHc(TradeContent.SadNoodleId);
bool hedgeGrewOnRedDay = false;
bool sawRedDay = false;
bool settleExact = true;
int bells = 0;
double stepSec = 30.0;
double capSeconds = (days + 2) * 1800.0;
double acc = 0;
string settleNote = "";
while (bells < days && acc < capSeconds)
{
    int bellsBefore = CloseBellCount(st.Market);
    st.Advance(stepSec);
    acc += stepSec;
    if (CloseBellCount(st.Market) <= bellsBefore) continue;
    bells++;
    // a bell rang inside this step: the settle booked on it
    string lastNoodle = null;
    foreach (var f in st.Content.Funnel)
        if (f.StartsWith("evt_sadness_yield@" + TradeContent.SadNoodleId + ":")) lastNoodle = f;
    if (lastNoodle == null) { settleExact = false; settleNote = "no noodle line on bell"; break; }
    string slice = lastNoodle.Substring(lastNoodle.IndexOf(':') + 1);
```

(middle pages omitted)

pages 31 - 254 of 284 omitted per first-30 + last-30 excerpt rule.

```
        /// <summary>Explicit event meeting (the offline summary face
        /// wiring point). Idempotent per id.</summary>
        public void UnlockEvent(string eid)
        {
            if (!string.IsNullOrEmpty(eid)) _eventAlbum.Add(eid);
        }

        /// <summary>evt_market_event@&lt;eid&gt;:... parser (the funnel
        /// line head law every event owner follows).</summary>
        public static bool TryParseEventId(string line, out string eid)
        {
            eid = null;
            if (string.IsNullOrEmpty(line) || !line.StartsWith("evt_market_event@")) return false;
            int from = "evt_market_event@".Length;
            int to = line.IndexOf(':', from);
            if (to < 0) to = line.Length;
            eid = line.Substring(from, to - from);
            return eid.Length > 0;
        }

        // ---- the today crazy index (M25: the whole city's mood gauge) ----

        /// <summary>The net effective day move over every listed meme
        /// stock (the wave plus the live event disturbances, clamped per
        /// board) - the mood input, pure derivation.</summary>
        public static int NetMoveBp(TradeMarket market)
        {
            int net = 0;
            foreach (var def in TradeData.Stocks())
            {
                var s = market.Stock(def.id);
                if (s != null) net += market.EffectiveDayBp(s);
            }
            return net;
        }

        /// <summary>The four-band closed set: frenzy over +800, greed
        /// over +300, panic under -300, laid-back between.</summary>
        public static string CrazyBandKeyOf(int netBp)
        {
            if (netBp >= CrazyFrenzyBandBp) return TradeTexts.KeyCrazyFrenzy;
            if (netBp >= CrazyGreedBandBp) return TradeTexts.KeyCrazyGreed;
            if (netBp <= -CrazyPanicBandBp) return TradeTexts.KeyCrazyPanic;
            return TradeTexts.KeyCrazyFlat;
        }

        /// <summary>The 0-100 gauge face: |net|/50 clamped (5000bp of
        /// aggregate city movement reads the full house).</summary>
        public static int CrazyIndexPctOf(int netBp)
        {
```

```
        int a = netBp < 0 ? -netBp : netBp;
        int pct = a / CrazyIndexDivisorBp;
        return pct > 100 ? 100 : pct;
    }

    /// <summary>The live triple (net, gauge, band) off the market.</summary>
    public (int netBp, int pct, string bandKey) CrazyIndex()
    {
        int net = NetMoveBp(_stage.Market);
        return (net, CrazyIndexPctOf(net), CrazyBandKeyOf(net));
    }

    // ---- the curve cells (the formation layer of every card) ----

    /// <summary>Pure level bucket: 1-5 over the series min-max
    /// (a flat series reads the mid bucket 3).</summary>
    public static int CurveCellTier(int assetsHc, int minHc, int maxHc)
    {
        if (maxHc <= minHc) return 3;
        int t = 1 + (assetsHc - minHc) * 4 / (maxHc - minHc);
        if (t < 1) t = 1;
        if (t > 5) t = 5;
        return t;
    }

    /// <summary>The formation cells for an asset series tail (the
    /// last maxCells points): each closed day one cell, the bucket
    /// tier paints the level, stack 1 (the day count rides the copy
    /// lines, the cells paint the shape).</summary>
    public static CardCell[] CellsFor(List<int> assetsSeries, int maxCells)
    {
        var cells = new List<CardCell>();
        if (assetsSeries == null || assetsSeries.Count == 0) return cells.ToArray();
        int min = assetsSeries[0], max = assetsSeries[0];
        foreach (var a in assetsSeries)
        {
            if (a < min) min = a;
            if (a > max) max = a;
        }
        int from = assetsSeries.Count > maxCells ? assetsSeries.Count - maxCells : 0;
        for (int i = from; i < assetsSeries.Count; i++)
            cells.Add(new CardCell { Tier = CurveCellTier(assetsSeries[i], min, max), Stack = 1 });
        return cells.ToArray();
    }

    /// <summary>This run's curve points (the card face: the journey
    /// belongs to the run, the history stays for the next one).</summary>
    public List<int> RunCurveAssets()
    {
        var list = new List<int>();
```

```
            foreach (var p in _curve)
                if (p.run == _runCount) list.Add(p.assetsHc);
            return list;
        }

        // ---- the three-in-one card (SYS08: curve + tier badge + crazy index) ----

        /// <summary>The run card spec: AutoCardComposer reuse - the title
        /// band, the assets/tier/crazy/collection copy lines, the share
        /// challenge footer, the kind/tier query payload and the curve
        /// formation layer (last 14 closed days).</summary>
        public ShareCard ComposeCard()
        {
            var m = _stage.Market;
            var crazy = CrazyIndex();
            var lines = new List<string>
            {
                TradeTexts.EnText(TradeTexts.KeyStatAssets) + ": " + HcStr(m.TotalAssetsHc),
                TradeTexts.EnText(TradeTexts.KeyStatTier) + ": " + TradeTexts.EnText(TradeTierGate.TierTitleKey(_stage.Gate.Tier)),
                TradeTexts.EnText(crazy.bandKey) + " " + crazy.pct + "/100",
                string.Format(TradeTexts.EnText(TradeTexts.KeyDailyReport), m.Day,
                    TradeTexts.EnText(TradeTexts.KeyAlbumTitle) + " " + AlbumTotalMet + "/" + AlbumTotalPool),
            };
            if (Board != null) lines.Add(Board.CardLine());   // the optional SYS06 slot (null = the untouched spec)
            var formation = CellsFor(RunCurveAssets(), CurveCellsMax);
            var card = AutoCardComposer.Card(CardTemplateStem,
                TradeTexts.EnText(TradeTexts.KeyCurveTitle),
                lines.ToArray(),
                TradeTexts.EnText(TradeTexts.KeyShareChallenge),
                "g10&kind=curve&tier=" + _stage.Gate.Tier + "&run=" + _runCount);
            card.Formation = formation;
            card.FormationCols = formation.Length;
            card.FormationRows = 1;
            return card;
        }

        /// <summary>The ending long card spec: the fund-manager summit
        /// title over the whole-run curve (the long image face). Null
        /// until the promotion spine reaches the fund-manager tier.</summary>
        public ShareCard ComposeEndingCard()
        {
            if (!_endingReached) return null;
            var m = _stage.Market;
            var crazy = CrazyIndex();
            var lines = new List<string>
            {
                TradeTexts.EnText(TradeTexts.KeyStatAssets) + ": " + HcStr(m.TotalAssetsHc),
                TradeTexts.EnText(TradeTexts.KeyStatTier) + ": " + TradeTexts.EnText(TradeTierGate.TierTitleKey(_stage.Gate.Tier)),
                TradeTexts.EnText(crazy.bandKey) + " " + crazy.pct + "/100",
                string.Format(TradeTexts.EnText(TradeTexts.KeyDailyReport), m.Day,
```

```
            TradeTexts.EnText(TradeTexts.KeyAlbumTitle) + " " + AlbumTotalMet + "/" + AlbumTotalPool),
    };
    if (Board != null) lines.Add(Board.CardLine());   // the optional SYS06 slot (null = the untouched spec)
    var formation = CellsFor(RunCurveAssets(), EndingCellsMax);
    var card = AutoCardComposer.Card(CardTemplateStem,
        TradeTexts.EnText(TradeTexts.KeyEndingTitle),
        lines.ToArray(),
        TradeTexts.EnText(TradeTexts.KeyShareChallenge),
        "g10&kind=ending&tier=" + _stage.Gate.Tier + "&run=" + _runCount);
    card.Formation = formation;
    card.FormationCols = formation.Length;
    card.FormationRows = 1;
    return card;
}

/// <summary>Composes and saves one card through the shared
/// ShareCardService: a successful compose books the evt_curve_share
/// telemetry line (kind/tier slices, the north-star share-rate
/// source); a failed compose books nothing.</summary>
public ComposeResult ComposeAndSave(ShareCardService svc, ShareCard card, string kind)
{
    var res = svc.Compose(card);
    if (res != null && res.Ok)
        _funnel.Add("evt_curve_share@" + kind + ":" + _stage.Gate.Tier);
    return res;
}

// ---- the ending and the second-run inheritance (SYS08) ----

/// <summary>Freezes the carry plan off the closing run: the 20%
/// floored gold, the kept trader ids (the core desk follows
/// along) and the run index the carry seeds. Read-only - the
/// world never moves here.</summary>
public NewRunCarry PlanNewRun()
{
    var carry = new NewRunCarry
    {
        GoldHc = _stage.Market.TotalAssetsHc * TradeData.InheritGoldPercent / 100,
        RunIndex = _runCount + 1,
    };
    if (_desk != null)
        foreach (var t in _desk.Collection)
            carry.TraderIds.Add(t.id);
    return carry;
}

/// <summary>Wires the next run onto the fresh stage: the carry
/// seeds the wallet, the kept traders land on the fresh desk's
/// collection (no draw, no ad burn), the seated inheritance
/// re-lives the first-trader experience (the T2 gate flag
```

```
        /// re-arms), the ladder restarts, the albums and the curve
        /// history persist and the ending re-arms for the new run.
        /// Returns the applied carry plan.</summary>
        public NewRunCarry StartNewRun(TradeStage freshStage, TradeDesk freshDesk)
        {
            var carry = PlanNewRun();
            freshStage.Market.BookGoldHc(carry.GoldHc);
            if (freshDesk != null && carry.TraderIds.Count > 0)
            {
                freshDesk.InheritFrom(carry.TraderIds);
                freshDesk.SyncRoster(1);
                if (freshDesk.Roster.Count > 0)
                    freshStage.Gate.RaiseFlag(TradeData.Tiers()[1].flagEventId);   // the inherited seat re-lives it
            }
            _stage = freshStage;
            _desk = freshDesk;
            _runCount++;
            _endingReached = false;
            _closeSeen = 0;
            _fillSeen = 0;
            _eventSeen = 0;
            _promoSeen = 0;
            _funnel.Add("evt_new_run@" + _runCount + ":" + carry.GoldHc);
            return carry;
        }

        static string HcStr(long hc)
            => (hc / 100.0).ToString("0.##", CultureInfo.InvariantCulture);

        /// <summary>Canonical fingerprint - the zero-RNG parity instrument
        /// (the showcase ledger plus the whole stage signature).</summary>
        public string Fingerprint()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("run").Append(_runCount);
            sb.Append("|end").Append(_endingReached ? 1 : 0);
            sb.Append("|curve").Append(_curve.Count);
            foreach (var p in _curve) sb.Append(';').Append(p.run).Append(',').Append(p.day).Append('@').Append(p.assetsHc);
            sb.Append("|alb").Append(_traderAlbum.Count).Append(',').Append(_stockAlbum.Count).Append(',').Append(_eventAlbum.Count);
            sb.Append("|fun").Append(_funnel.Count);
            foreach (var f in _funnel) sb.Append('#').Append(f);
            sb.Append("||").Append(_stage.OpeningSig());
            return sb.ToString();
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Runtime\Game\TradeStage.cs =====
// TradeStage.cs - the opening beat driver (G10-T005).
// Drives TradeMarket through the frozen red-line-11 opening script and
// stamps every promise on the sim clock: the K-line hook owns t0, the
```

```
// teaching order lands during the auction (pre-buried for the open print),
// the open bell rings at 8.00s and the fill fires the explosion triple, the
// opening bounce books the first coin within 1.5s of the fill with the
// coin-roll triple on one tick, then the all-in bro cameo (the first laugh)
// and the first market event card land inside the 10s window. Pure C#
// zero-RNG: the same script always fingerprints byte-identical.
using System.Collections.Generic;

namespace Sy.CrazyStocks
{
    public enum TradePhase
    {
        Hook,          // the K-line big screen + the limit-up red board caption
        AuctionPending, // the teaching order sits in the auction
        FirstFill,      // the open bell rang, the first fill printed
        TradingUp,      // the first coin rolled in
    }

    public sealed class TradeStage
    {
        // the frozen opening script (red-line-11 beat anchors)
        public const double OrderWindowStart = 3.0;  // T+3s: the order face goes live
        public const double TraderCameoAt = 9.0;     // T+9s: the all-in bro cameo (the first laugh)
        public const double MarketEventAt = 9.5;      // T+9.5s: the first market event card
        public const string TutorialStockId = "stk_leek_power";  // the T1 single main-board meme stock

        readonly TradeMarket _market;
        readonly TradeEventEngine _events;   // the T007 event stream rides the stage clock
        readonly TradeTierGate _gate;        // the T008 promotion spine (SYS05)
        readonly TradeIpoFund _ipoFund;      // the T009 tier-ticket desk (SYS07)
        readonly TradeBoard _board;         // the T013 boards face (SYS06, revisit-refresh law)
        readonly TradeContent _content;      // the T015 content trio (M16/M17/M23) rides the close bell
        readonly TradeTierContent _tierContent;   // the T016 quartet (M18/M19/M24/M25) rides the close bell, tier-gated
        readonly List<(string id, double t)> _stamps = new List<(string, double)>();
        double _t;
        int _assetHighWaterHc = TradeMarket.StartGoldHc;
        int _closeSeen;          // the session-mark scan pointer (the close-bell seal watch)
        bool _bellStamped;
        bool _firstFill;
        bool _firstCoin;
        bool _cameoDone;
        bool _eventDone;
        int _fillCountSeen;
        int _firstCoinAssetsBefore;

        TradeStage(TradeMarket market)
        {
            _market = market;
            _events = new TradeEventEngine(market, 1);
            _gate = new TradeTierGate();
```

```
            _ipoFund = new TradeIpoFund(market) { Gate = _gate };   // the tier-ticket desk wires onto the spine
            _board = new TradeBoard(market, _ipoFund);   // the boards ride the market + the SYS07 dragon feed (the host refreshes on
every visit)
            _content = new TradeContent(market);   // the content trio settles on the close bell (no stamp face: the opening sig
stays frozen)
            _tierContent = new TradeTierContent(market);   // the tier-content quartet settles on the same bell (steles / fortune /
city mood)
        }

        /// <summary>The fresh leek: the private stash lands in the wallet
        /// eight seconds before the opening bell, the board glows red.</summary>
        public static TradeStage NewLeek()
            => new TradeStage(new TradeMarket(TradeMarket.OpeningJoinAuctionSecond));

        // ---- live faces ----
        public double SimTime => _t;
        public TradePhase Phase
        {
            get
            {
                if (_firstCoin) return TradePhase.TradingUp;
                if (_firstFill) return TradePhase.FirstFill;
                if (TimeOf("evt_first_order") >= 0) return TradePhase.AuctionPending;
                return TradePhase.Hook;
            }
        }
        public TradeMarket Market => _market;
        public TradeEventEngine Events => _events;
        public TradeTierGate Gate => _gate;   // the promotion spine (the desk and UI faces wire onto it)
        public TradeIpoFund IpoFund => _ipoFund;   // the T009 tier-ticket desk (IPO / counter-bet / joint push / fund)
        public TradeBoard Board => _board;         // the T013 boards face (dragon + perf, Refresh() on every visit)
        public TradeContent Content => _content;   // the T015 content trio (sadness economy / intel desk / replay theater)
        public TradeTierContent TierContent => _tierContent;   // the T016 quartet (mystic plaque / stuck chronicle / leek rank /
city mania)
        public bool ModalActive => false;              // zero login wall, zero blocking modal (red line 11)
        public bool CanOrder => TimeOf("evt_first_order") < 0;   // the order face is live right after the hook
        public IReadOnlyList<(string id, double t)> Stamps => _stamps;

        /// <summary>First stamp time of an event id (-1 when never fired).</summary>
        public double TimeOf(string id)
        {
            for (int i = 0; i < _stamps.Count; i++)
                if (_stamps[i].id == id) return _stamps[i].t;
            return -1;
        }

        void Stamp(string id) => _stamps.Add((id, _t));

        /// <summary>Advances the opening clock: drives the market and
        /// fires the scheduled beats (open bell, first fill, first coin,
        /// trader cameo, market event card) on the frozen script.</summary>
        public void Advance(double dt)
        {
            if (dt <= 0) return;
```

```
if (TimeOf("evt_trade_hook") < 0)
    Stamp("evt_trade_hook");   // the hook owns t0 (K-line screen + red board + caption)

_t += dt;
_market.Advance(dt);

// the open bell crossing (the market clock owns the ring)
if (!_bellStamped)
{
    for (int i = 0; i < _market.SessionMarks.Count; i++)
    {
        if (_market.SessionMarks[i].sessionId == "open_bell")
        {
            Stamp("evt_open_bell");
            _bellStamped = true;
            break;
        }
    }
}

// the first fill: the explosion triple fires on the fill tick
if (!_firstFill && _market.Fills.Count > _fillCountSeen)
{
    _fillCountSeen = _market.Fills.Count;
    _firstFill = true;
    Stamp("evt_first_fill");
    Stamp("fill_visual");   // the fill explosion face
    Stamp("fill_sfx");      // the match chime trigger
    Stamp("fill_number");   // the wallet spend roll
}

// the first coin: the first total-assets high-water roll-up
// (the intraday peak face: the coin fires on the tick print
// that first crosses the high-water mark)
int peak = _market.MaxAssetsHc;
if (peak > _assetHighWaterHc)
{
    if (!_firstCoin)
    {
        _firstCoinAssetsBefore = _assetHighWaterHc;
        Stamp("evt_first_coin");
        Stamp("coin_visual");   // the coin-roll face
        Stamp("coin_sfx");     // the signature coin chime trigger
        Stamp("coin_number");  // the assets counter jump
        _firstCoin = true;
    }
    _assetHighWaterHc = peak;
}

// the first laugh: the all-in bro cameo with the attitude line
```

```
            if (!_cameoDone && _t >= TraderCameoAt)
            {
                Stamp("evt_trader_cameo");
                Stamp("evt_first_laugh");
                _cameoDone = true;
            }

            // the first market event card (the +2% teaching disturbance)
            if (!_eventDone && _t >= MarketEventAt)
            {
                _market.ApplyEventDisturbance(TutorialStockId, 2);
                Stamp("evt_market_event_card");
                _eventDone = true;
            }

            // the T007 event stream: the rotation slots and the chain
            // beats ride the same clock (first wave at the P50 anchor)
            _events.Advance(dt);

            // the T008 promotion spine: the sealed-board watch first (the
            // T3 lived experience), then the gate tick - a promotion
            // re-scopes the event pool and stamps the ceremony beats
            ObserveCloseBellSeals();
            _gate.Advance(dt);
            if (_gate.Tick(_market.TotalAssetsHc))
            {
                _events.SetTier(_gate.Tier);
                Stamp("evt_rank_up");
                Stamp(TradeTierGate.CeremonyAwardVisual);
                Stamp(TradeTierGate.CeremonyMcBroadcast);
            }
        }

        /// <summary>The close-bell seal watch (the T3 lived experience):
        /// a held stock sealing the limit-up board at the closing bell =
        /// the player rode the squad's board. E06 stays the post-promotion
        /// milestone celebration card; the flag comes from the market
        /// itself, so the gate chain never deadlocks on a T3-gated card.
        /// Raised once, idempotent on the flag face.</summary>
        void ObserveCloseBellSeals()
        {
            var marks = _market.SessionMarks;
            while (_closeSeen < marks.Count)
            {
                var m = marks[_closeSeen++];
                if (m.sessionId != "close_bell") continue;
                _ipoFund.OnCloseBell(m.day);   // the bell hands the tier-ticket settlements (T009) before the seal watch
                _content.OnCloseBell(m.day);   // the bell settles the content trio (T015: sadness yields + the replay theater, Mode-
gated)
                _tierContent.OnCloseBell(m.day, _gate.Tier);   // the bell beats the tier-content quartet (T016: steles / fortune /
city mood, tier-gated)
                foreach (var def in TradeData.Stocks())
```

```
                {
                    var s = _market.Stock(def.id);
                    if (s == null || s.shares <= 0) continue;
                    int limitBp = TradeData.BoardFor(def.boardId).limitPct * 100;
                    if (_market.EffectiveDayBp(s) >= limitBp)
                    {
                        _gate.RaiseFlag(TradeData.Tiers()[2].flagEventId);
                        return;
                    }
                }
            }
        }

        /// <summary>The teaching order: an all-in-flavored market buy of
        /// 90% of the stash on the tutorial stock (pre-buried in the
        /// auction, it matches at the opening print).</summary>
        public bool PlaceFirstOrder()
        {
            if (TimeOf("evt_trade_hook") < 0) return false;   // the hook owns t0
            if (TimeOf("evt_first_order") >= 0) return false;  // one teaching order
            var s = _market.Stock(TutorialStockId);
            int budget = _market.WalletHc * TradeMarket.TutorialBuyPermille / 1000;
            int shares = budget / s.priceHc;
            int ticket = _market.PlaceOrder(TutorialStockId, true, true, 0, shares);
            if (ticket <= 0) return false;
            Stamp("evt_first_order");
            return true;
        }

        public int FirstCoinJumpHc => _firstCoin ? _market.MaxAssetsHc - _firstCoinAssetsBefore : 0;
        public string CameoLine => TradeTexts.EnText(TradeTexts.KeyTraderAllInBro);

        /// <summary>Opening fingerprint: every stamp plus the market state
        /// plus the event stream plus the promotion spine - byte-identical
        /// across identical scripts.</summary>
        public string OpeningSig()
        {
            var sb = new System.Text.StringBuilder();
            foreach (var s in _stamps) sb.Append(s.id).Append('@').Append(s.t.ToString("0.###")).Append(';');
            sb.Append(_market.Fingerprint());
            sb.Append("||").Append(_events.Fingerprint());
            sb.Append("||").Append(_gate.Fingerprint());
            return sb.ToString();
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Runtime\Game\TradeTexts.cs =====
// TradeTexts.cs - the exchange copy law for Crazy Trade (G10-T004).
// English source strings live here as compiled code (the family source law:
// code strings stay English); the zh-CN localization rides a separate data
```

```
// file (Assets/_Game/Configs/Texts_zhCN.json - data, not source). Lookup
// follows the shared LocaleService contract: a missing key falls back to the
// key itself (machine-readable, never a crash), and a zh entry missing from
// the data file falls back to the English source. Zero scattered copy: every
// UI-facing sentence routes through these keys - DataSchemaCheck scans the
// runtime tree for stray CJK and proves the coverage law. The fictional-
// market disclaimer is a permanent resident key (compliance face, always on).
using System;
using System.Collections.Generic;
using Sy.Core;

namespace Sy.CrazyStocks
{
    public static class TradeTexts
    {
        // ---- key constants (the compiled copy surface) ----
        public const string KeyHookCaption = "hook_caption";
        public const string KeyMarketDisclaimer = "market_disclaimer";   // the permanent compliance face
        public const string KeyTier1 = "tier_1";
        public const string KeyTier2 = "tier_2";
        public const string KeyTier3 = "tier_3";
        public const string KeyTier4 = "tier_4";
        public const string KeyTier5 = "tier_5";
        public const string KeyStatAssets = "stat_assets";
        public const string KeyStatTier = "stat_tier";
        public const string KeyStatGold = "stat_gold";
        public const string KeyBoardMain = "board_main";
        public const string KeyBoardGem = "board_gem";
        public const string KeyBoardCrazy = "board_crazy";
        public const string KeyBoardNight = "board_night";
        public const string KeySessionAuction = "session_auction";
        public const string KeySessionOpenBell = "session_open_bell";
        public const string KeySessionMorning = "session_morning";
        public const string KeySessionLunch = "session_lunch";
        public const string KeySessionAfternoon = "session_afternoon";
        public const string KeySessionCloseBell = "session_close_bell";
        public const string KeyLimitUp = "limit_up";
        public const string KeyLimitDown = "limit_down";
        public const string KeyOrderPlace = "order_place";
        public const string KeyOrderFilled = "order_filled";
        public const string KeyFeeTrade = "fee_trade";
        public const string KeyIpoLuck = "ipo_luck";
        public const string KeyJointBoard = "joint_board";
        public const string KeyFundFee = "fund_fee";
        // ---- T009 tier-ticket seats: the IPO desk, the counter-bet, the
        // joint push and the fund faces (closed compiled set) ----
        public const string KeyIpoApply = "ipo_apply";
        public const string KeyIpoMiss = "ipo_miss";
        public const string KeyIpoBroke = "ipo_broke";
        public const string KeyIpoSurvive = "ipo_survive";
```

```
        public const string KeyCounterBet = "counter_bet";
        public const string KeyCounterWin = "counter_bet_win";
        public const string KeyCounterLose = "counter_bet_lose";
        public const string KeyGangUpTea = "gang_up_tea";
        public const string KeyDragonList = "dragon_list";
        public const string KeyFundIssue = "fund_issue";
        public const string KeyFundDanmaku1 = "fund_danmaku_1";
        public const string KeyFundDanmaku2 = "fund_danmaku_2";
        public const string KeyFundDanmaku3 = "fund_danmaku_3";
        public const string KeyIcebergDesk = "iceberg_desk";
        public const string KeyBigLotSelfLift = "big_lot_self_lift";
        public const string KeyIcebergLanded = "iceberg_landed";
        public const string KeyIcebergBuilt = "iceberg_built";
        public const string KeyRankUp = "rank_up";
        public const string KeyRankUpMc = "rank_up_mc";
        public const string KeyTraderAllInBro = "trader_all_in_bro";
        public const string KeyTraderSteadyWang = "trader_steady_wang";
        public const string KeyTraderCounterLi = "trader_counter_li";
        public const string KeyTraderGrannyInfo = "trader_granny_info";
        public const string KeyTraderBoardSnatcher = "trader_board_snatcher";
        public const string KeyTraderTopEscapeSis = "trader_top_escape_sis";
        public const string KeyTraderLeekTrainer = "trader_leek_trainer";
        public const string KeyAlbumTitle = "album_title";
        public const string KeyCurveTitle = "curve_title";
        public const string KeyOfflineSummary = "offline_summary";
        public const string KeyOfflineDouble = "offline_double";
        public const string KeyOvernightGap = "overnight_gap";
        public const string KeyCarryLabel = "carry_label";
        public const string KeyNewJourney = "new_journey";
        public const string KeyNewJourneyConfirm = "new_journey_confirm";
        public const string KeyShareChallenge = "share_challenge";
        public const string KeyDailyReport = "daily_report";
        // ---- schema v2 seats: the 8th/9th trader cards + the E01-E20 event
        // title closed set (every event row carries exactly one of these) ----
        public const string KeyTraderNeverWrongDing = "trader_never_wrong_ding";
        public const string KeyTraderSweeperMonk = "trader_sweeper_monk";
        public const string KeyEventTitleE01 = "event_title_e01";
        public const string KeyEventTitleE02 = "event_title_e02";
        public const string KeyEventTitleE03 = "event_title_e03";
        public const string KeyEventTitleE04 = "event_title_e04";
        public const string KeyEventTitleE05 = "event_title_e05";
        public const string KeyEventTitleE06 = "event_title_e06";
        public const string KeyEventTitleE07 = "event_title_e07";
        public const string KeyEventTitleE08 = "event_title_e08";
        public const string KeyEventTitleE09 = "event_title_e09";
        public const string KeyEventTitleE10 = "event_title_e10";
        public const string KeyEventTitleE11 = "event_title_e11";
        public const string KeyEventTitleE12 = "event_title_e12";
        public const string KeyEventTitleE13 = "event_title_e13";
        public const string KeyEventTitleE14 = "event_title_e14";
```

```
        public const string KeyEventTitleE15 = "event_title_e15";
        public const string KeyEventTitleE16 = "event_title_e16";
        public const string KeyEventTitleE17 = "event_title_e17";
        public const string KeyEventTitleE18 = "event_title_e18";
        public const string KeyEventTitleE19 = "event_title_e19";
        public const string KeyEventTitleE20 = "event_title_e20";
        // ---- T011 showcase seats: the three album tabs, the crazy-index
        // four-band closed set and the fund-manager summit ending ----
        public const string KeyAlbumTraders = "album_traders";
        public const string KeyAlbumStocks = "album_stocks";
        public const string KeyAlbumEvents = "album_events";
        public const string KeyCrazyFlat = "crazy_flat";
        public const string KeyCrazyGreed = "crazy_greed";
        public const string KeyCrazyFrenzy = "crazy_frenzy";
        public const string KeyCrazyPanic = "crazy_panic";
        public const string KeyEndingTitle = "ending_title";
        // ---- T013 board seats: the two-board titles, the player seat,
        // the card slot line and the hot-money crew signature closed set
        // (the actor-troupe cameo law) ----
        public const string KeyBoardTitle = "board_title";
        public const string KeyBoardDragon = "board_dragon";
        public const string KeyBoardPerf = "board_perf";
        public const string KeyBoardYou = "board_you";
        public const string KeyBoardCardLine = "board_card_line";
        public const string KeyOppSquadAdmiral = "opp_squad_admiral";
        public const string KeyOppFloorSky = "opp_floor_sky";
        public const string KeyOppAuctionBro = "opp_auction_bro";
        public const string KeyOppOneDayTourist = "opp_one_day_tourist";
        // ---- T014 M15 LIVE dual-time faces: the compliance wording (the
        // market calls itself the live tape, never a real exchange) and
        // the three clock-mode labels ----
        public const string KeyLiveTape = "live_tape";
        public const string KeyClockLive = "clock_mode_live";
        public const string KeyClockReplay = "clock_mode_replay";
        public const string KeyClockClosed = "clock_mode_closed";
        // ---- T015 content-trio seats: the three sadness assets (M16),
        // the three guru schools with the fee/scam/degree faces (M17)
        // and the replay theater pool (M23: the title, the six rotating
        // templates, the rehearsal skip line) ----
        public const string KeySadAssetNoodle = "sad_asset_noodle";
        public const string KeySadAssetCafe = "sad_asset_cafe";
        public const string KeySadAssetFireworks = "sad_asset_fireworks";
        public const string KeySadYieldLine = "sad_yield_line";
        public const string KeyIntelSchoolDance = "intel_school_dance";
        public const string KeyIntelSchoolMystic = "intel_school_mystic";
        public const string KeyIntelSchoolTechnical = "intel_school_technical";
        public const string KeyIntelFeeLine = "intel_fee_line";
        public const string KeyIntelScamEvent = "intel_scam_event";
        public const string KeyAchievementTrickedThrice = "achievement_tricked_thrice";
        public const string KeyReplayTitle = "replay_title";
```

```
        public const string KeyReplayTpl1 = "replay_tpl_1";
        public const string KeyReplayTpl2 = "replay_tpl_2";
        public const string KeyReplayTpl3 = "replay_tpl_3";
        public const string KeyReplayTpl4 = "replay_tpl_4";
        public const string KeyReplayTpl5 = "replay_tpl_5";
        public const string KeyReplayTpl6 = "replay_tpl_6";
        public const string KeyReplaySkipLine = "replay_skip_line";
        // the T016 tier-content quartet keys (M18/M19/M24/M25)
        public const string KeyMysticSchoolFengShui = "mystic_school_feng_shui";
        public const string KeyMysticSchoolTaoFlow = "mystic_school_tao_flow";
        public const string KeyMysticSchoolKoiPalace = "mystic_school_koi_palace";
        public const string KeyMysticFortuneFeng1 = "mystic_fortune_feng_1";
        public const string KeyMysticFortuneFeng2 = "mystic_fortune_feng_2";
        public const string KeyMysticFortuneFeng3 = "mystic_fortune_feng_3";
        public const string KeyMysticFortuneTao1 = "mystic_fortune_tao_1";
        public const string KeyMysticFortuneTao2 = "mystic_fortune_tao_2";
        public const string KeyMysticFortuneTao3 = "mystic_fortune_tao_3";
        public const string KeyMysticFortuneKoi1 = "mystic_fortune_koi_1";
        public const string KeyMysticFortuneKoi2 = "mystic_fortune_koi_2";
        public const string KeyMysticFortuneKoi3 = "mystic_fortune_koi_3";
        public const string KeyMysticCompliance = "mystic_compliance";
        public const string KeyUnstuckCertTitle = "unstuck_cert_title";
        public const string KeyUnstuckCertStock = "unstuck_cert_stock";
        public const string KeyUnstuckCertDays = "unstuck_cert_days";
        public const string KeyUnstuckCertDepth = "unstuck_cert_depth";
        public const string KeyUnstuckCertDay = "unstuck_cert_day";
        public const string KeyLeekBronze = "leek_bronze";
        public const string KeyLeekSilver = "leek_silver";
        public const string KeyLeekGold = "leek_gold";
        public const string KeyLeekPlatinum = "leek_platinum";
        public const string KeyLeekDiamond = "leek_diamond";
        public const string KeyLeekStar = "leek_star";
        public const string KeyLeekKing = "leek_king";
        public const string KeyLeekSupreme = "leek_supreme";
        public const string KeyLeekBadgeLine = "leek_badge_line";
        public const string KeyCityMoodFrenzy1 = "city_mood_frenzy_1";
        public const string KeyCityMoodFrenzy2 = "city_mood_frenzy_2";
        public const string KeyCityMoodFrenzy3 = "city_mood_frenzy_3";
        public const string KeyCityMoodGreed1 = "city_mood_greed_1";
        public const string KeyCityMoodGreed2 = "city_mood_greed_2";
        public const string KeyCityMoodGreed3 = "city_mood_greed_3";
        public const string KeyCityMoodPanic1 = "city_mood_panic_1";
        public const string KeyCityMoodPanic2 = "city_mood_panic_2";
        public const string KeyCityMoodPanic3 = "city_mood_panic_3";
        public const string KeyCityMoodFlat1 = "city_mood_flat_1";
        public const string KeyCityMoodFlat2 = "city_mood_flat_2";
        public const string KeyCityMoodFlat3 = "city_mood_flat_3";
        public const string KeyCityManiaLine = "city_mania_line";
        public const string KeyManiaCompliance = "mania_compliance";
        // ---- T023a M21 drama-theater seats: the blind-box face line, the
```

```
        // fictional regulator letter and the th01-th08 pool title closed
        // set (the letter never fines - pure theater, zero real-technique
        // wording, the compliance law) ----
        public const string KeyTheaterBlindBox = "theater_blind_box";
        public const string KeyTheaterRegulatorLetter = "theater_regulator_letter";
        public const string KeyTheaterTitleTh01 = "theater_title_th01";
        public const string KeyTheaterTitleTh02 = "theater_title_th02";
        public const string KeyTheaterTitleTh03 = "theater_title_th03";
        public const string KeyTheaterTitleTh04 = "theater_title_th04";
        public const string KeyTheaterTitleTh05 = "theater_title_th05";
        public const string KeyTheaterTitleTh06 = "theater_title_th06";
        public const string KeyTheaterTitleTh07 = "theater_title_th07";
        public const string KeyTheaterTitleTh08 = "theater_title_th08";
        // ---- T024a M22 fund-sentiment seats: the three holder-steadying
        // means (the letter, the roadshow, the red packet), the tide face
        // and the rt01-rt06 redemption-tide comedy pool titles (the tide
        // is comedy, never a death spiral - the fund always rebuilds,
        // zero real-technique wording, the compliance law) ----
        public const string KeyFundLetter = "fund_letter";
        public const string KeyFundRoadshow = "fund_roadshow";
        public const string KeyFundRedpacket = "fund_redpacket";
        public const string KeyFundTide = "fund_tide";
        public const string KeyFundTideTitleRt01 = "fund_tide_title_rt01";
        public const string KeyFundTideTitleRt02 = "fund_tide_title_rt02";
        public const string KeyFundTideTitleRt03 = "fund_tide_title_rt03";
        public const string KeyFundTideTitleRt04 = "fund_tide_title_rt04";
        public const string KeyFundTideTitleRt05 = "fund_tide_title_rt05";
        public const string KeyFundTideTitleRt06 = "fund_tide_title_rt06";

        /// <summary>English source table (compiled - the source-of-law face).</summary>
        public static readonly Dictionary<string, string> En = new Dictionary<string, string>
        {
            { KeyHookCaption, "WELCOME TO CRAZY TRADE - YOUR FUND-MANAGER LIFE STARTS NOW" },
            { KeyMarketDisclaimer, "FICTIONAL MARKET FOR ENTERTAINMENT - NOT INVESTMENT ADVICE" },
            { KeyTier1, "THE LEEK RETAIL" },
            { KeyTier2, "MID-TIER" },
            { KeyTier3, "BIG PLAYER" },
            { KeyTier4, "HOT MONEY" },
            { KeyTier5, "FUND MANAGER" },
            { KeyStatAssets, "ASSETS" },
            { KeyStatTier, "TIER" },
            { KeyStatGold, "GOLD" },
            { KeyBoardMain, "MAIN BOARD" },
            { KeyBoardGem, "GEM MEME BOARD" },
            { KeyBoardCrazy, "CRAZY BOARD" },
            { KeyBoardNight, "NIGHT-SNACK BOARD" },
            { KeySessionAuction, "CALL AUCTION" },
            { KeySessionOpenBell, "OPENING BELL" },
            { KeySessionMorning, "MORNING SESSION" },
            { KeySessionLunch, "LUNCH BREAK" },
```

```
        { KeySessionAfternoon, "AFTERNOON SESSION" },
        { KeySessionCloseBell, "CLOSING BELL" },
        { KeyLimitUp, "LIMIT UP!" },
        { KeyLimitDown, "LIMIT DOWN..." },
        { KeyOrderPlace, "PLACE ORDER" },
        { KeyOrderFilled, "FILLED!" },
        { KeyFeeTrade, "FEE: {0}%" },
        { KeyIpoLuck, "ALLOTTED! RARER THAN A LOTTERY WIN" },
        { KeyJointBoard, "JOINT BOARD PUSH" },
        { KeyFundFee, "FUND FEE: {0}%/DAY" },
        { KeyIpoApply, "IPO SUBSCRIPTION - SLOTS BOOKED, PRAY FOR LUCK" },
        { KeyIpoMiss, "MISSED - 0.3% IS 0.3%" },
        { KeyIpoBroke, "BROKE BELOW ISSUE - THE HYPE PREMIUM EVAPORATES" },
        { KeyIpoSurvive, "HELD THROUGH THE BREAK - THE LEEK LIVES" },
        { KeyCounterBet, "COUNTER-BET - RIDE AGAINST THE CALL" },
        { KeyCounterWin, "THE CALL WAS WRONG AGAIN - DOUBLE THE STAKE" },
        { KeyCounterLose, "THE CALL WAS RIGHT FOR ONCE - THE STAKE IS GONE" },
        { KeyGangUpTea, "THE REGULATOR POURS TEA (PURELY FICTIONAL)" },
        { KeyDragonList, "THE DRAGON LIST - TODAY'S GANG-UP SEALS" },
        { KeyFundIssue, "ISSUE THE CRAZY FUND - THE HOLDERS ARRIVE" },
        { KeyFundDanmaku1, "PRAISE THE MANAGER - FLAT IS THE NEW UP" },
        { KeyFundDanmaku2, "MANAGER, SHOW UP - THE COMMENT SECTION MISSES YOU" },
        { KeyFundDanmaku3, "ALL-IN ON THE FUND - WHAT COULD GO WRONG" },
        { KeyIcebergDesk, "ICEBERG DESK - SLICE THE MEGALOT, SLIP IN QUIET" },
        { KeyBigLotSelfLift, "THE MEGALOT LIFTED THE PRICE AGAINST YOU - THE DESK CHUCKLES" },
        { KeyIcebergLanded, "ONE MORE QUIET SLICE LANDED - STILL UNDER THE RADAR" },
        { KeyIcebergBuilt, "POSITION BUILT BELL BY BELL - THE MARKET NEVER NOTICED" },
        { KeyRankUp, "TIER UP! {0} UNLOCKED" },
        { KeyRankUpMc, "CRAZY FINANCE CHANNEL:" },
        { KeyTraderAllInBro, "ALL-IN BRO - ALL-IN IS AN ATTITUDE" },
        { KeyTraderSteadyWang, "STEADY WANG - HALF POSITION, FULL NOON NAP" },
        { KeyTraderCounterLi, "COUNTER LI - TRUST LI, THEN DO THE OPPOSITE" },
        { KeyTraderGrannyInfo, "GRANNY INFO - THE SQUARE-DANCE WIRE" },
        { KeyTraderBoardSnatcher, "BOARD SNATCHER - LIMIT-UP OR NOTHING" },
        { KeyTraderTopEscapeSis, "TOP-ESCAPE SIS - SELLING EARLY IS ALSO PROFIT" },
        { KeyTraderLeekTrainer, "LEEK TRAINER - LOSSES BECOME EXPERIENCE" },
        { KeyAlbumTitle, "MEME ALBUM" },
        { KeyCurveTitle, "MY ASSET CURVE" },
        { KeyOfflineSummary, "WHILE YOU WERE GONE..." },
        { KeyOfflineDouble, "DOUBLE IT?" },
        { KeyOvernightGap, "OVERNIGHT GAP: {0}%" },
        { KeyCarryLabel, "NEXT-RUN CARRY: {0}" },
        { KeyNewJourney, "START A NEW RUN" },
        { KeyNewJourneyConfirm, "TAP AGAIN - THE CARRY SEEDS YOUR NEXT LEEK" },
        { KeyShareChallenge, "CAN YOUR CURVE BEAT MINE?" },
        { KeyDailyReport, "DAY {0}: {1}" },
        { KeyTraderNeverWrongDing, "NEVER-WRONG DING - IT'S NOT ME, IT'S THE MARKET" },
        { KeyTraderSweeperMonk, "THE SWEEPING MONK - I DON'T TRADE, I JUST SWEEP" },
        { KeyEventTitleE01, "METAVERSE PANCAKE WINS A TEN-MILLION FUNDING ROUND" },
        { KeyEventTitleE02, "LEEK POWER ANNUAL REPORT BLOWS UP" },
```

```
        { KeyEventTitleE03, "SQUARE-DANCE AUDIO WINS A GOVERNMENT ORDER" },
        { KeyEventTitleE04, "A HOT-MONEY WHALE DROPS BY" },
        { KeyEventTitleE05, "SYSTEM MAINTENANCE FALSE ALARM" },
        { KeyEventTitleE06, "THE LIMIT-UP SQUAD ASSEMBLES" },
        { KeyEventTitleE07, "IPO DAY - THE ALLOTMENT CEREMONY" },
        { KeyEventTitleE08, "GANG-UP SUCCESS - A SMALL MEME STOCK HITS THE BOARD" },
        { KeyEventTitleE09, "THE FUND-HOLDER COMMENT GALA" },
        { KeyEventTitleE10, "THE OVERNIGHT GAP SURPRISE" },
        { KeyEventTitleE11, "THE LIGHTS-OUT NOODLE FESTIVAL" },
        { KeyEventTitleE12, "THE ROOFTOP VIEW CAFE OPENS" },
        { KeyEventTitleE13, "THE 88TH-FLOOR SENTRY TOWER OPENS" },
        { KeyEventTitleE14, "HALFWAY MONUMENT VS FLOOR-PRICE MEMORIAL" },
        { KeyEventTitleE15, "CRAZY CIRCUIT-BREAKER DAY" },
        { KeyEventTitleE16, "THE ANALYST TRIO TV DEBATE" },
        { KeyEventTitleE17, "THE CROW-MOUTH HITS THE STREET" },
        { KeyEventTitleE18, "THE HOARDING HAMSTER REVEALED" },
        { KeyEventTitleE19, "THE MYSTERIOUS NORTHBOUND FIGURE" },
        { KeyEventTitleE20, "THE BREAK-EVEN MEMORIAL HALL" },
        { KeyAlbumTraders, "ALBUM: THE TRADER DESK" },
        { KeyAlbumStocks, "ALBUM: MEME STOCKS" },
        { KeyAlbumEvents, "ALBUM: MARKET EVENTS" },
        { KeyCrazyFlat, "CRAZY INDEX: LAID-BACK" },
        { KeyCrazyGreed, "CRAZY INDEX: GREED" },
        { KeyCrazyFrenzy, "CRAZY INDEX: FRENZY" },
        { KeyCrazyPanic, "CRAZY INDEX: PANIC" },
        { KeyEndingTitle, "FUND-MANAGER!" },
        { KeyBoardTitle, "THE CRAZY EXCHANGE BOARDS" },
        { KeyBoardDragon, "DRAGON LIST - TODAY'S TOP MOVERS" },
        { KeyBoardPerf, "PERF BOARD - TODAY'S DAY RETURNS" },
        { KeyBoardYou, "YOU - THE LEEK IN THE ARENA" },
        { KeyBoardCardLine, "TODAY: PERF RANK #{0}/{1} - TOP MOVER {2} {3}" },
        { KeyOppSquadAdmiral, "SQUAD ADMIRAL - SEE YOU ON THE BOARD" },
        { KeyOppFloorSky, "FLOOR-TO-SKY MANIAC - THE FLOOR IS MY LAUNCHPAD" },
        { KeyOppAuctionBro, "AUCTION BRO - THE OPENING PRINT OBEYS ME" },
        { KeyOppOneDayTourist, "ONE-DAY TOURIST - IN TODAY, GONE TOMORROW" },
        { KeyLiveTape, "LIVE TAPE" },
        { KeyClockLive, "LIVE - THE REAL SESSION WINDOW, SAME MOMENT EVERYWHERE" },
        { KeyClockReplay, "REPLAY - TODAY'S 30-MINUTE REHEARSAL, FREE" },
        { KeyClockClosed, "MARKET CLOSED - THE HOLIDAY STAGE IS ON" },
        { KeySadAssetNoodle, "NOODLE BAR - THE WHOLE-CITY EATS-NOODLES INDEX FUND" },
        { KeySadAssetCafe, "ROOFTOP CAFE - THE WIND-BLOWER VIEW INDEX" },
        { KeySadAssetFireworks, "FIREWORKS STAND - THE TEARS CELEBRATION FUND" },
        { KeySadYieldLine, "SADNESS YIELD: {0} {1}bp (TAPE {2}bp)" },
        { KeyIntelSchoolDance, "SQUARE-DANCE WIRE - THE MORNING SCOOP, RUMOR-GRADE" },
        { KeyIntelSchoolMystic, "DREAM-CHART MASTERS - THE MYSTIC READERS, RUMOR-GRADE" },
        { KeyIntelSchoolTechnical, "NEVER-WRONG DESK - THE TECHNICAL BUREAU, RUMOR-GRADE" },
        { KeyIntelFeeLine, "SUBSCRIPTION: {0}/DAY - ACCURACY {1} PER MILLE, ENTERTAINMENT ONLY" },
        { KeyIntelScamEvent, "THE CALL WAS A RUMOR - THE DESK EATS NOODLES, THE RUN LIVES ON" },
        { KeyAchievementTrickedThrice, "TRICKED THRICE - THE LEEK EARNS ITS DEGREE" },
        { KeyReplayTitle, "TONIGHT'S CLOSING REPLAY THEATER" },
```

```
            { KeyReplayTpl1, "TODAY THE LEEK LANDED {0} FILLS - THE CHANNEL SALUTES THE EFFORT" },
            { KeyReplayTpl2, "TOP MOVER {1} SWUNG {2}BP - THE EXPERTS ARE SPEECHLESS" },
            { KeyReplayTpl3, "THE WALLET CLOSED AT {3} - NOODLES FOR EVERYONE" },
            { KeyReplayTpl4, "SADNESS INDEX {4}BP - THE NOODLE BAR HIRES NIGHT STAFF" },
            { KeyReplayTpl5, "THE GUEST DESK SAYS: {5}" },
            { KeyReplayTpl6, "{5} - SAME TIME TOMORROW, THE TAPE NEVER SLEEPS" },
            { KeyReplaySkipLine, "REPLAY SESSION - A FREE REHEARSAL, NO FORMAL REPLAY TONIGHT" },
            { KeyMysticSchoolFengShui, "FENG-SHUI WHEEL SECT - THE PLAQUE OF SPINNING LUCK" },
            { KeyMysticSchoolTaoFlow, "TAO FLOW GATE - THE PLAQUE OF EFFORTLESS DRIFT" },
            { KeyMysticSchoolKoiPalace, "KOI PALACE - THE PLAQUE OF THE LUCKY FISH" },
            { KeyMysticFortuneFeng1, "A FINE DAY TO LIE FLAT - ORDERS FILL SLOWER, BUT THE INSOMNIA EVENT CANNOT TOUCH YOU (COMEDY
ONLY, ZERO EDGE)" },
            { KeyMysticFortuneFeng2, "THE WHEEL SPINS A RED TALE - SELL NOTHING, BUY NOTHING, THE UNIVERSE SHRUGS (COMEDY ONLY, ZERO
EDGE)" },
            { KeyMysticFortuneFeng3, "LUCK SMELLS LIKE NOODLES TODAY - WEAR THE SEATBELT BACKWARDS, STYLE POINTS ONLY (COMEDY ONLY,
ZERO EDGE)" },
            { KeyMysticFortuneTao1, "THE TAO SAYS DRIFT - THE TAPE WIGGLES, YOU NAP THROUGH IT (COMEDY ONLY, ZERO EDGE)" },
            { KeyMysticFortuneTao2, "FLOW LIKE WATER, TRADE LIKE A SNOOZING CAT - THE MARKET OWES YOU NOTHING, THE PLAQUE OWES YOU A
SMILE (COMEDY ONLY, ZERO EDGE)" },
            { KeyMysticFortuneTao3, "TODAY'S LUCKY DIRECTION IS DOWN THE STREET FOR NOODLES - THE FORTUNE NEVER TOUCHES YOUR ORDERS
(COMEDY ONLY, ZERO EDGE)" },
            { KeyMysticFortuneKoi1, "THE KOI NODS ONCE - SELLING FEARS YOU TODAY, BUYING FEARS YOU MORE (COMEDY ONLY, ZERO EDGE)" },
            { KeyMysticFortuneKoi2, "A QUARTER MOON OVER THE EXCHANGE - WEAR SOCKS THAT CLASH, FORTUNE FOLLOWS THE BOLD DRESSER
(COMEDY ONLY, ZERO EDGE)" },
            { KeyMysticFortuneKoi3, "THE LUCKY FISH BLOWS A BUBBLE AT THE TICKER - IT MEANS NOTHING AND THAT IS THE JOKE (COMEDY
ONLY, ZERO EDGE)" },
            { KeyMysticCompliance, "MYSTIC PLAZA IS PURE COMEDY - IT NEVER TOUCHES BUY OR SELL OUTCOMES" },
            { KeyUnstuckCertTitle, "BREAK-EVEN!" },
            { KeyUnstuckCertStock, "{0}" },
            { KeyUnstuckCertDays, "STUCK FOR {0} DAYS" },
            { KeyUnstuckCertDepth, "DEEPEST DIVE: {0}BP" },
            { KeyUnstuckCertDay, "CLAWED BACK ON DAY {0}" },
            { KeyLeekBronze, "BRONZE LEEK" },
            { KeyLeekSilver, "SILVER LEEK" },
            { KeyLeekGold, "GOLD LEEK" },
            { KeyLeekPlatinum, "PLATINUM LEEK" },
            { KeyLeekDiamond, "DIAMOND LEEK" },
            { KeyLeekStar, "STAR LEEK" },
            { KeyLeekKing, "KING LEEK" },
            { KeyLeekSupreme, "SUPREME LEEK" },
            { KeyLeekBadgeLine, "LEEK RANK: {0}" },
            { KeyCityMoodFrenzy1, "NPC: THE SQUID VENDOR QUIT SELLING SQUID AND BOUGHT THREE BOARDS" },
            { KeyCityMoodFrenzy2, "NPC: THE GRANNY DANCE TROUPE CHANGED THEIR ANTHEM TO THE TICKER SONG" },
            { KeyCityMoodFrenzy3, "NPC: THE NOODLE BAR PUT UP A FULL HOUSE SIGN AT DAWN" },
            { KeyCityMoodGreed1, "NPC: THE BARBER ASKS EVERY CLIENT ONE QUESTION - ANY TIPS" },
            { KeyCityMoodGreed2, "NPC: THE TAXI FLEET REROUTED EVERY FARE PAST THE EXCHANGE" },
            { KeyCityMoodGreed3, "NPC: THE BUBBLE TEA SHOP SELLS A LIMIT-UP FLAVOR" },
            { KeyCityMoodPanic1, "NPC: THE SQUID VENDOR IS BACK - BUSINESS IS BOOMING AGAIN" },
            { KeyCityMoodPanic2, "NPC: THE ROOFTOP CAFE RUNS OUT OF SEATS WITH A VIEW" },
            { KeyCityMoodPanic3, "NPC: THE FIREWORKS STAND RESTOCKED THREE TIMES BY NOON" },
            { KeyCityMoodFlat1, "NPC: THE SWEEPING MONK SWEPT THE SAME SQUARE TWICE - NOTHING HAPPENED" },
            { KeyCityMoodFlat2, "NPC: THE PIGEONS FILED A COMPLAINT ABOUT THE BORING TAPE" },
            { KeyCityMoodFlat3, "NPC: THE NOODLE BAR SELLS THE USUAL - THE USUAL IS FINE" },
            { KeyCityManiaLine, "THE CRAZY GAUGE HITS FULL - THE WHOLE CITY DANCES AT ONCE" },
            { KeyManiaCompliance, "THE CITY MOOD IS PURE THEATER - NEVER AN INVESTMENT REFERENCE" },
            { KeyTheaterBlindBox, "DRAMA THEATER - EVERY MISS ROLLS A COMEDY BLIND BOX" },
            { KeyTheaterRegulatorLetter, "REGULATOR WANG'S LETTER (FICTIONAL) - TEA AND MELON SEEDS, ZERO FINE" },
```

```
        { KeyTheaterTitleTh01, "REGULATOR WANG SENDS A FICTIONAL WARNING LETTER" },
        { KeyTheaterTitleTh02, "THE HOT-MONEY CREW COUNTER-SNIPES THE PUSH" },
        { KeyTheaterTitleTh03, "THE MELON-EATING CROWD GATHERS AROUND THE TAPE" },
        { KeyTheaterTitleTh04, "THE PUSH MISSES BUT THE FAME LANDS ANYWAY" },
        { KeyTheaterTitleTh05, "THE FORUM LIGHTS UP IN A FLAME WAR" },
        { KeyTheaterTitleTh06, "THE SQUARE-DANCE TROUPE CASCADES INTO THE LOBBY" },
        { KeyTheaterTitleTh07, "THE NOODLE BAR SERVES THE SPECIAL COMBO" },
        { KeyTheaterTitleTh08, "CRAZY FINANCE CHANNEL AIRS A SPECIAL REPORT" },
        { KeyFundLetter, "REASSURANCE LETTER - THE MANAGER PINKY PROMISES" },
        { KeyFundRoadshow, "LIVESTREAM ROADSHOW - TRUST THE PROCESS, IGNORE THE CHART" },
        { KeyFundRedpacket, "GOLD RED PACKET - SORRY IN COINS, TIDE STOPPED INSTANTLY" },
        { KeyFundTide, "REDEMPTION TIDE - THE HOLDERS MARCH OUT (COMEDY, NEVER A CRASH)" },
        { KeyFundTideTitleRt01, "THE HOLDERS EXIT UNDER A FAREWELL BANNER" },
        { KeyFundTideTitleRt02, "THE FORUM THREAD IS A REDEMPTION EXPRESS" },
        { KeyFundTideTitleRt03, "THE LEEK TRAINER CALLS IT A STRATEGIC WITHDRAWAL" },
        { KeyFundTideTitleRt04, "THE BANK QUEUE STRETCHES TO THE NOODLE BAR" },
        { KeyFundTideTitleRt05, "CRAZY CHANNEL AIRS: WHERE DID THE HOLDERS GO" },
        { KeyFundTideTitleRt06, "THE SQUARE-DANCE TROUPE EXITS IN PARADE FORMATION" },
    };

    /// <summary>English lookup (missing key falls back to the key itself
    /// - machine-readable, never a crash).</summary>
    public static string EnText(string key)
        => En.TryGetValue(key, out var v) && !string.IsNullOrEmpty(v) ? v : key;

    /// <summary>Parses the zh-CN data file payload into a key-to-text
    /// map (values must be non-empty strings; anything else drops the
    /// entry so the lookup degrades to English, never to garbage).</summary>
    public static Dictionary<string, string> ParseZh(string json)
    {
        var map = new Dictionary<string, string>();
        if (string.IsNullOrEmpty(json)) return map;
        try
        {
            if (!(MiniJson.Deserialize(json) is Dictionary<string, object> d)) return map;
            foreach (var kv in d)
                if (!string.IsNullOrEmpty(kv.Key) && kv.Value is string s && !string.IsNullOrEmpty(s))
                    map[kv.Key] = s;
        }
        catch
        {
            // corrupt data file = empty map = English fallback face
            return new Dictionary<string, string>();
        }
        return map;
    }

    /// <summary>LocaleService-compatible lookup delegate over a parsed
    /// zh map: zh-CN hits read the data file, everything else (and any
    /// miss) reads the English source. Wire-ready for the T005 UI face.</summary>
```

```
        public static Func<string, string, string> LookupWithZh(Dictionary<string, string> zh)
        {
            return (lang, key) =>
            {
                if (lang == "zh-CN" && zh != null && zh.TryGetValue(key, out var v) && !string.IsNullOrEmpty(v))
                    return v;
                return EnText(key);
            };
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Runtime\Game\TradeTierContent.cs =====
// TradeTierContent.cs - the T016 tier-gated content quartet (M18/M19/M24/
// M25): the born-sign mystic plaque, the stuck chronicle, the leek rank
// ladder and the city-mania dashboard upgrade. Four light mechanisms
// riding the existing faces (the halving law): the close bell hands the
// daily beat exactly like the content trio, the copy rides the M11 text
// engine (compiled English source + zh data file), and every value
// derives zero-RNG off the frozen wave math (the same day always prints
// the same numbers).
//   M18 mystic plaque: the player hangs one mystic school in the hall -
//   each day prints one fortune line with a nonsense comedy buff baked
//   in ("a fine day to lie flat: orders fill slower, but the insomnia
//   event cannot touch you"). Comedy only, zero numeric edge: the
//   plaque stamps a telemetry line and touches no price, no wallet,
//   no order - the compliance note is compiled dead into the copy.
//   M19 stuck chronicle: every held position under its cost base gets
//   a stele carved (stock id + first-stuck day + running day count +
//   depth in bp); the day the position claws back over its cost the
//   stele moves into the break-even memorial (the M12 face as data),
//   the whole city firesworks, and the unstuck certificate composes
//   as a shareable card (the F4 canary face).
//   M24 leek rank: a pure-derivation ladder of self-mocking titles
//   (bronze leek .. supreme leek) off the behavior score (fills +
//   scam losses + break-even memorials) - titles only, zero numeric
//   gain, the badge line feeds the share-card stock.
//   M25 mania dashboard: the T011 crazy index upgraded into the whole
//   city's mood stage - the frozen four-band set stays, each band
//   rotates three NPC mood lines zero-RNG by day, and a full gauge
//   (100/100) fires the city-wide mania event. Pure theater, never
//   a stock tip - the compliance note is compiled dead.
using System;
using System.Collections.Generic;
using System.Text;
using Sy.Social;

namespace Sy.CrazyStocks
{
    public sealed class TradeTierContent
    {
```

```
    // ---- v1 wiring (sweep-tunable; the doc pack owns the faces) ----
    public const int MysticSchoolCount = 3;
    public const int MysticFortunePool = 3;         // three fortune lines per school, zero-RNG day rotation
    public const int UnstuckMemorialCap = 30;        // the break-even memorial hall keeps the recent stones
    public const int ManiaFullPct = 100;             // the full gauge fires the city event
    public const int LeekScamWeight = 5;              // every rumor-school scam feeds the leek score
    public const int LeekMemorialWeight = 10;        // every unstuck stone feeds the leek score

    public const string SchoolFengShuiId = "feng_shui_fortune";
    public const string SchoolTaoFlowId = "tao_flow_gate";
    public const string SchoolKoiPalaceId = "koi_palace";

    // the leek ladder (M24): frozen thresholds - pure titles, zero numeric gain
    public static readonly int[] LeekThresholds = { 0, 30, 80, 180, 400, 800, 1600, 3200 };
    public static readonly string[] LeekTitleKeys =
    {
        TradeTexts.KeyLeekBronze, TradeTexts.KeyLeekSilver, TradeTexts.KeyLeekGold, TradeTexts.KeyLeekPlatinum,
        TradeTexts.KeyLeekDiamond, TradeTexts.KeyLeekStar, TradeTexts.KeyLeekKing, TradeTexts.KeyLeekSupreme,
    };

    /// <summary>A mystic school (M18): the hall plaque - three fortune
    /// lines with the comedy buff baked in, zero-RNG rotation by day.
    /// Comedy only, zero numeric edge (the compliance note is dead
    /// law: the plaque never touches buy or sell outcomes).</summary>
    public sealed class MysticSchoolDef
    {
        public string id = "";
        public string nameKey = "";
        public int seed;
        public string[] fortuneKeys = new string[MysticFortunePool];
    }

    /// <summary>A stuck stele (M19): carved on the first close under
    /// the cost base, the day count grows each closing bell and the
    /// depth keeps the deepest underwater print seen so far (in bp)
    /// - the stone never softens a dive it already recorded.</summary>
    public sealed class StuckStele
    {
        public string stockId = "";
        public int sinceDay;
        public int days;
        public int depthBp;
    }

    /// <summary>A break-even memorial stone (M19): the stele that
    /// clawed back over its cost base - the M12 memorial hall as a
    /// data face (the unstuck certificate composes off it).</summary>
    public sealed class UnstuckMemorial
    {
        public string stockId = "";
```

```
            public int days;
            public int depthBp;
            public int day;
        }

        static readonly MysticSchoolDef[] MysticTable =
        {
            new MysticSchoolDef
            {
                id = SchoolFengShuiId, nameKey = TradeTexts.KeyMysticSchoolFengShui, seed = 99101,
                fortuneKeys = new[] { TradeTexts.KeyMysticFortuneFeng1, TradeTexts.KeyMysticFortuneFeng2,
TradeTexts.KeyMysticFortuneFeng3 },
            },
            new MysticSchoolDef
            {
                id = SchoolTaoFlowId, nameKey = TradeTexts.KeyMysticSchoolTaoFlow, seed = 99107,
                fortuneKeys = new[] { TradeTexts.KeyMysticFortuneTao1, TradeTexts.KeyMysticFortuneTao2,
TradeTexts.KeyMysticFortuneTao3 },
            },
            new MysticSchoolDef
            {
                id = SchoolKoiPalaceId, nameKey = TradeTexts.KeyMysticSchoolKoiPalace, seed = 99113,
                fortuneKeys = new[] { TradeTexts.KeyMysticFortuneKoi1, TradeTexts.KeyMysticFortuneKoi2,
TradeTexts.KeyMysticFortuneKoi3 },
            },
        };

        readonly TradeMarket _market;
        readonly List<string> _funnel = new List<string>();
        readonly Dictionary<string, StuckStele> _steles = new Dictionary<string, StuckStele>();
        readonly List<UnstuckMemorial> _memorial = new List<UnstuckMemorial>();
        string _schoolId = "";

        public TradeTierContent(TradeMarket market)
        {
            _market = market;
        }

        // ---- live faces ----
        public IReadOnlyList<string> Funnel => _funnel;
        public IReadOnlyDictionary<string, StuckStele> Steles => _steles;
        public IReadOnlyList<UnstuckMemorial> Memorial => _memorial;
        public string SelectedSchool => _schoolId;
        public static IReadOnlyList<MysticSchoolDef> MysticSchools => MysticTable;

        public static MysticSchoolDef MysticSchool(string id)
        {
            foreach (var s in MysticTable) if (s.id == id) return s;
            return null;
        }

        // ---- M18: the born-sign mystic plaque ----
```

```
        /// <summary>Hangs one school's plaque in the hall (M18). The
        /// plaque stamps nothing but its own face: no price, no wallet,
        /// no order ever moves here (the compliance note is dead law -
        /// the mystic never touches buy or sell outcomes).</summary>
        public bool SelectSchool(string id)
        {
            if (MysticSchool(id) == null) return false;
            _schoolId = id;
            Book("evt_mystic_plaque", id, "hung");
            return true;
        }

        /// <summary>The day's fortune line key (zero-RNG pool rotation):
        /// the same school on the same day always prints the same line.
        /// The comedy buff is baked into the copy (net-zero theater -
        /// "orders fill slower, but the insomnia event cannot touch you"
        /// is the joke, never a stat).</summary>
        public static string FortuneKeyOf(string schoolId, int day)
        {
            var s = MysticSchool(schoolId);
            if (s == null) return "";
            return s.fortuneKeys[(int)(TradeData.MixPublic(s.seed, (uint)day) % MysticFortunePool)];
        }

        /// <summary>The composed fortune line (empty until a plaque
        /// hangs; the View reads this on the close-bell beat).</summary>
        public string DailyFortuneLine(int day)
        {
            if (string.IsNullOrEmpty(_schoolId)) return "";
            string key = FortuneKeyOf(_schoolId, day);
            string line = TradeTexts.EnText(key);
            Book("evt_mystic_fortune", _schoolId, "d" + day + ":" + key);
            return line;
        }

        // ---- M19: the stuck chronicle ----

        /// <summary>The stuck law: a held position under its cost base
        /// is stuck (value = shares x last print against the position
        /// cost base). The depth is the underwater percentage in bp.</summary>
        public static int StuckDepthBp(int shares, int priceHc, int costHc)
        {
            if (shares <= 0 || costHc <= 0) return 0;
            long value = (long)shares * priceHc;
            long cost = costHc;
            if (value >= cost) return 0;
            return (int)((cost - value) * 10000 / cost);
        }

        /// <summary>The close-bell beat (wired by TradeStage exactly like
```

```
        /// the content trio): the stuck chronicle scans every held
        /// position (tier T2+, first stuck carve starts the chronicle),
        /// the mystic plaque prints the day's fortune and the mania
        /// dashboard rolls the city mood (both tier T1+). The M24 leek
        /// rank is a pure derivation - no bell face, the View reads it
        /// any time.</summary>
        public void OnCloseBell(int day, int tier)
        {
            if (tier >= 2)
            {
                foreach (var def in TradeData.Stocks())
                {
                    var s = _market.Stock(def.id);
                    if (s == null || s.shares <= 0 || s.costHc <= 0) continue;
                    int depth = StuckDepthBp(s.shares, s.priceHc, s.costHc);
                    if (depth > 0)
                    {
                        if (!_steles.TryGetValue(def.id, out var stele))
                        {
                            stele = new StuckStele { stockId = def.id, sinceDay = day, days = 1, depthBp = depth };
                            _steles[def.id] = stele;
                            Book("evt_stuck_carved", def.id, "first:d" + day + ":" + depth + "bp");
                        }
                        else
                        {
                            stele.days++;
                            if (depth > stele.depthBp) stele.depthBp = depth;   // the stone keeps the deepest print
                            Book("evt_stuck_carved", def.id, "d" + day + ":" + stele.days + "d:" + depth + "bp");
                        }
                    }
                    else if (_steles.TryGetValue(def.id, out var stele))
                    {
                        // the claw-back day: the stele moves into the memorial,
                        // the whole city fires fireworks, the certificate arms
                        _steles.Remove(def.id);
                        var stone = new UnstuckMemorial { stockId = def.id, days = stele.days, depthBp = stele.depthBp, day = day };
                        _memorial.Add(stone);
                        if (_memorial.Count > UnstuckMemorialCap) _memorial.RemoveAt(0);
                        Book("evt_unstuck_fireworks", def.id, "d" + day + ":" + stele.days + "d:" + stele.depthBp + "bp");
                    }
                }
            }
            if (tier >= 1)
            {
                if (!string.IsNullOrEmpty(_schoolId))
                    DailyFortuneLine(day);
                var face = CrazyIndexFace();
                string moodKey = CityMoodKeyOf(face.bandKey, day);
                Book("evt_city_mood", face.bandKey, "d" + day + ":" + moodKey + ":" + face.pct);
                if (face.pct >= ManiaFullPct)
```

```
                Book("evt_city_mania", "pct" + ManiaFullPct, "d" + day + ":" + face.netBp);
        }
    }

    /// <summary>The unstuck certificate card (M19): the break-even
    /// memorial composes into a shareable card - the stock, the
    /// stuck day count, the deepest underwater print and the
    /// claw-back day (the F4 canary face). AutoCardComposer reuse,
    /// no formation layer - the certificate is a copy face.</summary>
    public ShareCard ComposeUnstuckCard(UnstuckMemorial stone)
    {
        if (stone == null) return null;
        var lines = new List<string>
        {
            string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertStock), stone.stockId),
            string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertDays), stone.days),
            string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertDepth), stone.depthBp),
            string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertDay), stone.day),
            LeekBadgeLine(),
        };
        return AutoCardComposer.Card(TradeShowcase.CardTemplateStem,
            TradeTexts.EnText(TradeTexts.KeyUnstuckCertTitle),
            lines.ToArray(),
            TradeTexts.EnText(TradeTexts.KeyShareChallenge),
            "g10&kind=unstuck&days=" + stone.days + "&depth=" + stone.depthBp + "&day=" + stone.day);
    }

    // ---- M24: the leek rank ladder ----

    /// <summary>The behavior score (pure derivation, zero state):
    /// the fill count plus the scam losses and the break-even
    /// memorials - every number re-derives off the books, so the
    /// ladder can never drift from the actual play.</summary>
    public int LeekScore(int scamTimes)
        => _market.Fills.Count + scamTimes * LeekScamWeight + _memorial.Count * LeekMemorialWeight;

    /// <summary>The rank index off the frozen threshold table (0 =
    /// bronze leek .. 7 = supreme leek). Titles only - the ladder
    /// hands out no numeric gain (the compliance law of M24).</summary>
    public static int LeekRankIndex(int score)
    {
        int idx = 0;
        for (int i = 0; i < LeekThresholds.Length; i++)
            if (score >= LeekThresholds[i]) idx = i;
        return idx;
    }

    public static string LeekTitleKeyOf(int idx)
        => LeekTitleKeys[idx];
```

```
        /// <summary>The badge line the share-card stock consumes (M24:
        /// the badge is card material, the ladder never touches the
        /// game numbers).</summary>
        public string LeekBadgeLine()
            => string.Format(TradeTexts.EnText(TradeTexts.KeyLeekBadgeLine),
TradeTexts.EnText(LeekTitleKeyOf(LeekRankIndex(LeekScore(0)))));

        // ---- M25: the city-mania dashboard upgrade ----

        /// <summary>The whole market's net effective day move (bp) - the
        /// same sum the T011 crazy index folds (zero RNG, the same
        /// clock always prints the same gauge).</summary>
        public int MarketNetBp()
        {
            long net = 0;
            foreach (var def in TradeData.Stocks())
            {
                var s = _market.Stock(def.id);
                if (s == null) continue;
                net += _market.EffectiveDayBp(s);
            }
            return (int)net;
        }

        /// <summary>The dashboard face: the net, the 0-100 gauge and the
        /// frozen four-band key (the T011 law reused verbatim - the
        /// upgrade adds the stage, never rewrites the bands).</summary>
        public (int netBp, int pct, string bandKey) CrazyIndexFace()
        {
            int net = MarketNetBp();
            return (net, TradeShowcase.CrazyIndexPctOf(net), TradeShowcase.CrazyBandKeyOf(net));
        }

        /// <summary>The city mood line key for a band on a day (M25):
        /// three NPC mood lines per band, zero-RNG rotation by day -
        /// the same city on the same day always speaks the same mood.</summary>
        public static string CityMoodKeyOf(string bandKey, int day)
        {
            string[] pool;
            if (bandKey == TradeTexts.KeyCrazyFrenzy) pool = new[] { TradeTexts.KeyCityMoodFrenzy1, TradeTexts.KeyCityMoodFrenzy2,
TradeTexts.KeyCityMoodFrenzy3 };
            else if (bandKey == TradeTexts.KeyCrazyGreed) pool = new[] { TradeTexts.KeyCityMoodGreed1, TradeTexts.KeyCityMoodGreed2,
TradeTexts.KeyCityMoodGreed3 };
            else if (bandKey == TradeTexts.KeyCrazyPanic) pool = new[] { TradeTexts.KeyCityMoodPanic1, TradeTexts.KeyCityMoodPanic2,
TradeTexts.KeyCityMoodPanic3 };
            else pool = new[] { TradeTexts.KeyCityMoodFlat1, TradeTexts.KeyCityMoodFlat2, TradeTexts.KeyCityMoodFlat3 };
            return pool[(int)(TradeData.MixPublic(55501, (uint)day) % pool.Length)];
        }

        /// <summary>Canonical fingerprint - the zero-RNG parity
        /// instrument for the tier-content stream (the same script
        /// over the same days is always byte-identical).</summary>
        public string Fingerprint()
        {
```

```
            var sb = new StringBuilder();
            sb.Append("school").Append(_schoolId);
            foreach (var f in _funnel) sb.Append(';').Append(f);
            foreach (var kv in _steles) sb.Append("|stele:").Append(kv.Key).Append('=')
                .Append(kv.Value.sinceDay).Append('/').Append(kv.Value.days).Append('/').Append(kv.Value.depthBp);
            foreach (var m in _memorial) sb.Append("|stone:").Append(m.stockId).Append('=')
                .Append(m.days).Append('/').Append(m.depthBp).Append('@').Append(m.day);
            return sb.ToString();
        }

        void Book(string evt, string id, string slice)
        {
            _funnel.Add(evt + "@" + id + ":" + slice);
        }
    }
}
// ===== FILE: Assets\_Game\Scripts\Runtime\Game\TradeTierGate.cs =====
// TradeTierGate.cs - the five-tier promotion spine (G10-T008, SYS05).
// The M04 growth spine consumes the frozen TradeData tier table: every tier
// is a dual-track gate - the meme asset threshold AND a lived-experience
// flag together, because pure money never buys a tier (a tier is a new
// gameplay ticket, never a numeric wall; the idle face unlocks on arrival,
// never blocks). The gate rises at most one tier per tick so every
// promotion owns its pixel award ceremony - the award stage, the fanfare
// and the Crazy Finance Channel broadcast each book their own beat, and
// the evt_rank_up telemetry line lands with the tier and unlock slices.
// The flag sources are the owning systems: the desk raises the
// first-trader flag on the first hire (T006), the stage raises the
// limit-up squad flag when a held stock seals the board at the bell (the
// lived T3 experience; E06 stays the post-promotion milestone card), and
// the IPO / joint-push systems raise the T4/T5 flags when they land
// (T009). The new-run reset is the inheritance law: the spine restarts
// at the leek retail tier with every flag cleared (only the opening
// stake re-arms). Pure C# zero-RNG: the same asset timeline and the same
// flags always promote on the same ticks.
using System.Collections.Generic;
using System.Text;

namespace Sy.CrazyStocks
{
    public sealed class TradeTierGate
    {
        // the ceremony beats (the pixel award show the UI face consumes)
        public const string CeremonyAwardVisual = "award_visual";
        public const string CeremonyAwardSfx = "award_sfx";
        public const string CeremonyMcBroadcast = "mc_broadcast";

        static readonly string[] CeremonyBeats = { CeremonyAwardVisual, CeremonyAwardSfx, CeremonyMcBroadcast };

        readonly List<string> _funnel = new List<string>();
```

```
        readonly List<string> _beats = new List<string>();
        readonly HashSet<string> _flags = new HashSet<string>();
        readonly List<(int tier, double at)> _promotions = new List<(int tier, double at)>();
        int _tier = 1;
        double _t;

        public TradeTierGate()
        {
            // T1 is held from the opening stake: the flag row of the first
            // gate reads the frozen table (zero drift by construction)
            _flags.Add(TradeData.Tiers()[0].flagEventId);
        }

        // ---- live faces ----
        public int Tier => _tier;
        public double SimTime => _t;
        public IReadOnlyList<string> Funnel => _funnel;
        public IReadOnlyList<string> Beats => _beats;
        public IReadOnlyList<(int tier, double at)> Promotions => _promotions;
        public int FlagCount => _flags.Count;
        public bool FlagOf(string flagEventId) => _flags.Contains(flagEventId);

        /// <summary>Advances the spine clock (promotion stamps ride it).</summary>
        public void Advance(double dt)
        {
            if (dt > 0) _t += dt;
        }

        /// <summary>Raises a lived-experience flag (the owning systems
        /// call this: the desk hire, the sealed board, the IPO face).
        /// Idempotent - an experience counts once, whatever re-triggers it.</summary>
        public void RaiseFlag(string flagEventId)
        {
            if (!string.IsNullOrEmpty(flagEventId)) _flags.Add(flagEventId);
        }

        /// <summary>The pure five-gate solve: the highest tier whose whole
        /// chain is satisfied - every asset threshold met in gold hundredths
        /// AND every flag lived, walking the frozen table in order (a broken
        /// link caps the ladder; the chain is monotonic by freeze). Money
        /// alone never solves a gate whose flag is missing.</summary>
        public static int SolveTier(long assetsHc, IEnumerable<string> flags)
        {
            var lived = flags as ICollection<string> ?? new HashSet<string>(flags);
            int solved = 1;
            foreach (var def in TradeData.Tiers())
            {
                bool assetsOk = assetsHc >= def.assetGate * 100;   // gold -> hundredths
                bool flagOk = lived.Contains(def.flagEventId);
                if (assetsOk && flagOk) solved = def.tier;
```

```
            else break;
        }
        return solved;
    }

    /// <summary>The live solve over this gate's flags.</summary>
    public int SolvedTier(long assetsHc) => SolveTier(assetsHc, _flags);

    /// <summary>The promotion tick: rises at most one tier per call
    /// (every promotion owns its own ceremony beat - a 100x asset
    /// jump cascades over consecutive ticks, never stacks), books the
    /// evt_rank_up telemetry line plus the channel broadcast line, and
    /// never demotes on a drawdown. Returns true when a promotion
    /// fired on this tick.</summary>
    public bool Tick(long assetsHc)
    {
        int next = _tier + 1;
        if (next > TradeData.TierCount) return false;
        var def = TradeData.Tiers()[next - 1];
        if (assetsHc < def.assetGate * 100) return false;         // the money track
        if (!_flags.Contains(def.flagEventId)) return false;      // the lived track
        _tier = next;
        _promotions.Add((next, _t));
        _funnel.Add("evt_rank_up@t" + next + ":" + def.unlockId);
        _funnel.Add("evt_rank_up_mc@t" + next);
        foreach (var b in CeremonyBeats) _beats.Add(b + "@t" + next);
        return true;
    }

    /// <summary>The tier title key (the ceremony copy face).</summary>
    public static string TierTitleKey(int tier)
    {
        switch (tier)
        {
            case 1: return TradeTexts.KeyTier1;
            case 2: return TradeTexts.KeyTier2;
            case 3: return TradeTexts.KeyTier3;
            case 4: return TradeTexts.KeyTier4;
            default: return TradeTexts.KeyTier5;
        }
    }

    /// <summary>The broadcast sentence the MC face reads on a
    /// promotion: the channel banner plus the tier-up line over the
    /// tier title (compiled English copy, zero CJK by law).</summary>
    public static string BroadcastLineOf(int tier)
        => TradeTexts.EnText(TradeTexts.KeyRankUpMc) + " "
           + string.Format(TradeTexts.EnText(TradeTexts.KeyRankUp), TradeTexts.EnText(TierTitleKey(tier)));

    /// <summary>The new-run reset (the inheritance law): the spine
```

```
        /// restarts at the leek retail tier with every flag cleared - only
        /// the opening stake re-arms (the albums and the curve history
        /// persist elsewhere, the spine never carries over).</summary>
        public void ResetForNewRun()
        {
            _tier = 1;
            _flags.Clear();
            _flags.Add(TradeData.Tiers()[0].flagEventId);
        }

        /// <summary>Canonical fingerprint - the zero-RNG parity instrument
        /// for the spine (rides the stage opening signature).</summary>
        public string Fingerprint()
        {
            var sb = new StringBuilder();
            sb.Append("tier").Append(_tier)
              .Append("|flags").Append(_flags.Count)
              .Append("|prom").Append(_promotions.Count)
              .Append("|beats").Append(_beats.Count)
              .Append("|fun").Append(_funnel.Count);
            foreach (var p in _promotions) sb.Append(";t").Append(p.tier).Append('@').Append(p.at.ToString("0.###"));
            return sb.ToString();
        }
    }
}
```